Optimization Theory and Combinatorics

Lecture 1 – Thursday May 8, 2014

Outline

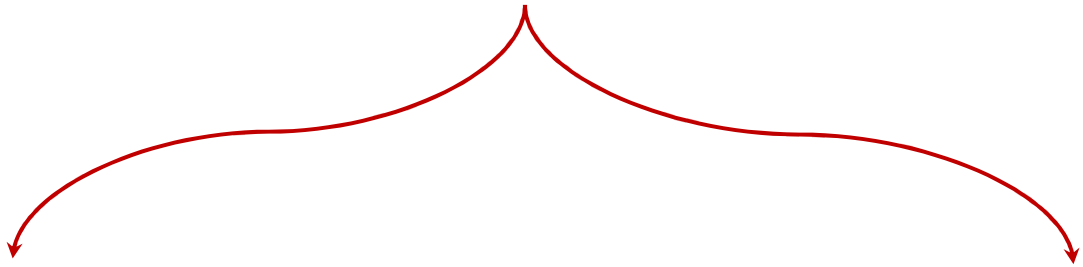
- Introduction to Optimization
- Optimization and Operations Research
- Optimization Problem
- Optimization Problem Classifications
- Optimization Methods
- Combinatorics
- Computational Complexity [For Reading]

Outline

- **Introduction to Optimization**
- Optimization and Operations Research
- Optimization Problem
- Optimization Problem Classifications
- Optimization Methods
- Combinatorics
- Computational Complexity [For Reading]

Introduction to Optimization

Computational Problems



Decision problems

A problem with a "yes" or "no" answer

Optimization problems

A computational problem in which the object is to **find the best of all possible solutions**. (i.e. find a solution in the feasible region which has the minimum or maximum value of the objective function.)

Introduction to Optimization

- **Computational Problems**

  Convert Optimization Problems into equivalent Decision Problems:

  - What is the optimal value?
  - Is there a feasible solution to the problem with an objective function value equal to or superior to a specified threshold?

Introduction to Optimization

Optimization is a term used to refer to a branch of computational science concerned with finding the **"best" solution** to a problem, possibly subject to a set of constraints.

"**best**" refers to **an acceptable (or satisfactory)** solution.

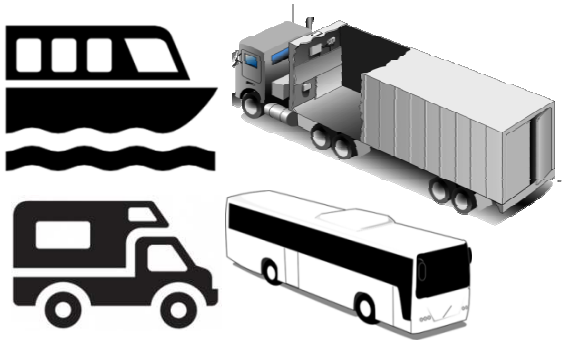
- **Types of solutions**
  - ◇ A **feasible solution** satisfies all constraints.
  - ◇ An **optimal solution** is feasible and provides the best objective function value.
  - ◇ A **near-optimal solution** is feasible and provides a superior objective function value, but not necessarily the best.

Introduction to Optimization

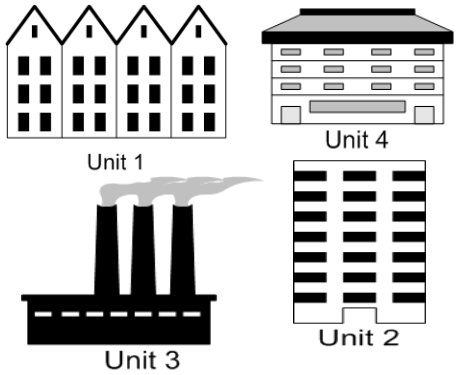
- Optimization is the act of obtaining the **best result** under given circumstances.
- The ultimate goal of any decision is either to **minimize the effort/cost required** or to **maximize the desired benefit**.
- Since the **effort required** or the **benefit desired** in most of practical situation can be expressed as a function of certain decision variables, **optimization** can be defined as the process of **finding the conditions** that give the **maximum or minimum value of a function**.

Introduction to Optimization

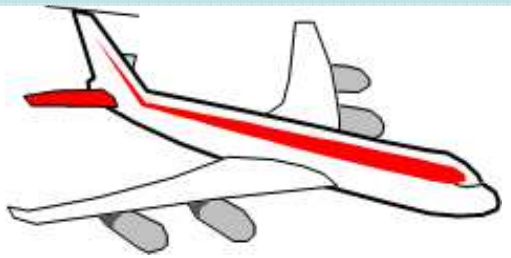
- Optimization is everywhere



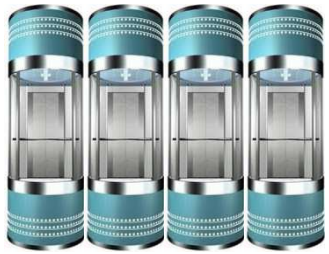
Vehicle routing



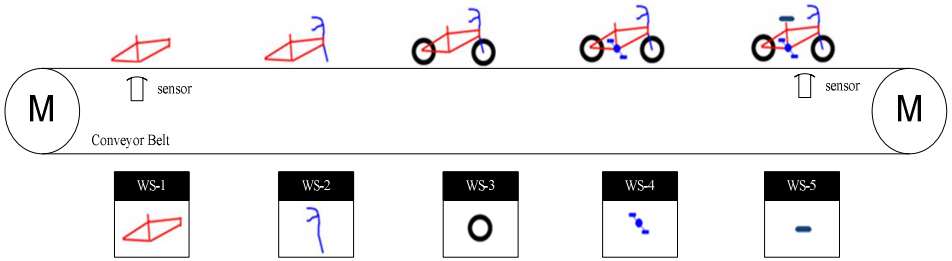
Plant layout



Airline scheduling

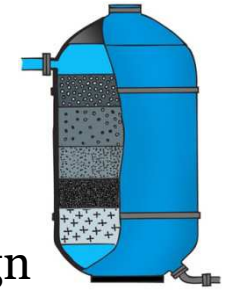


Elevator dispatching



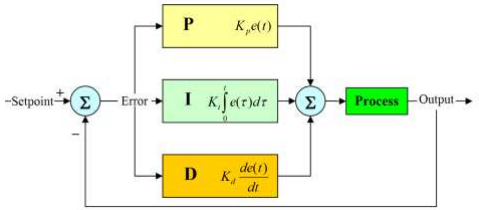
Assembly line balancing

Railway scheduling

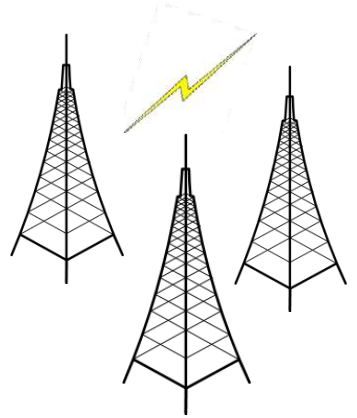


Filter design

Path Planning



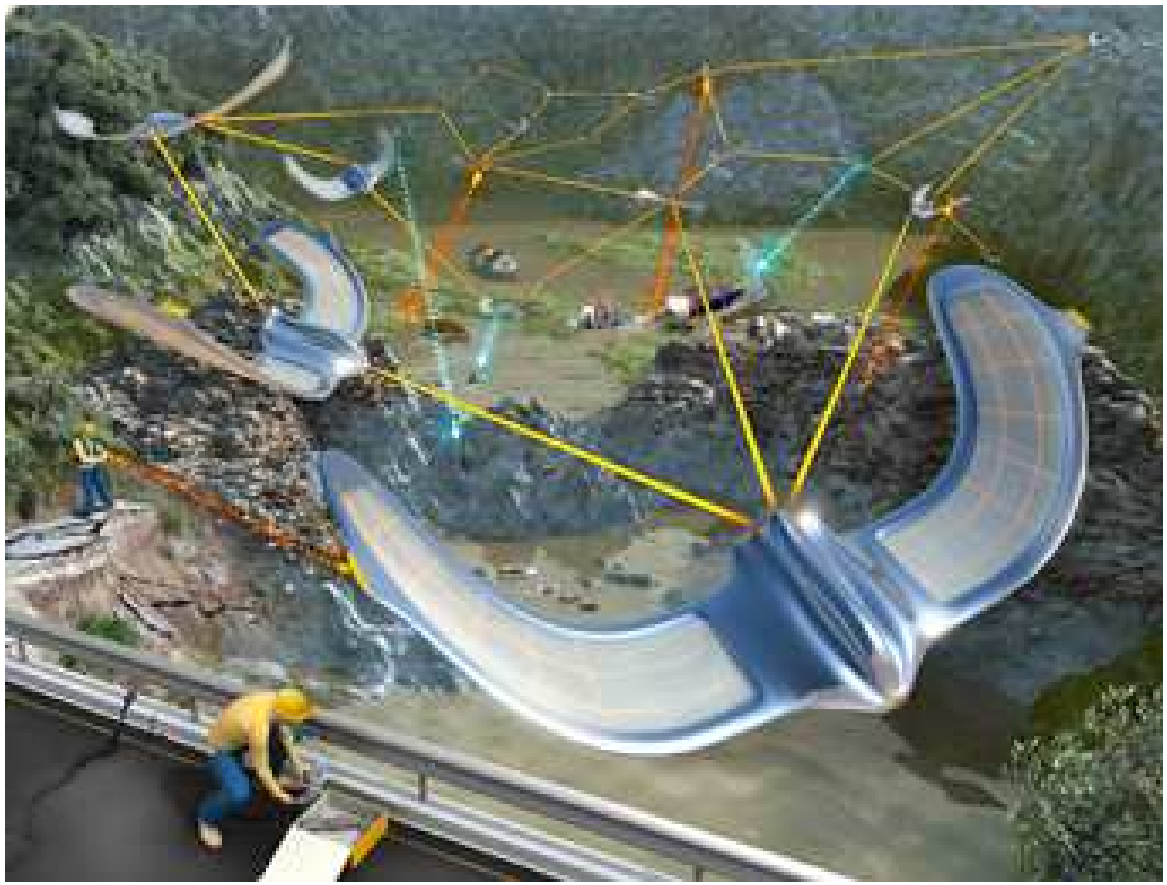
PID Auto-Tuning



Cell assignment

Introduction to Optimization

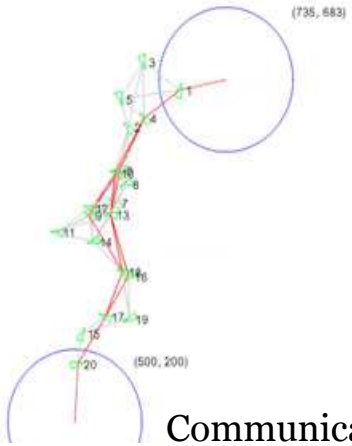
- Optimization is everywhere
  - ◇ Unmanned Aerial/Ground/Marine Vehicles (UXVs)



UAVs Deployment

Maximize:

- Area Coverage
- Object Coverage
- Radio Coverage



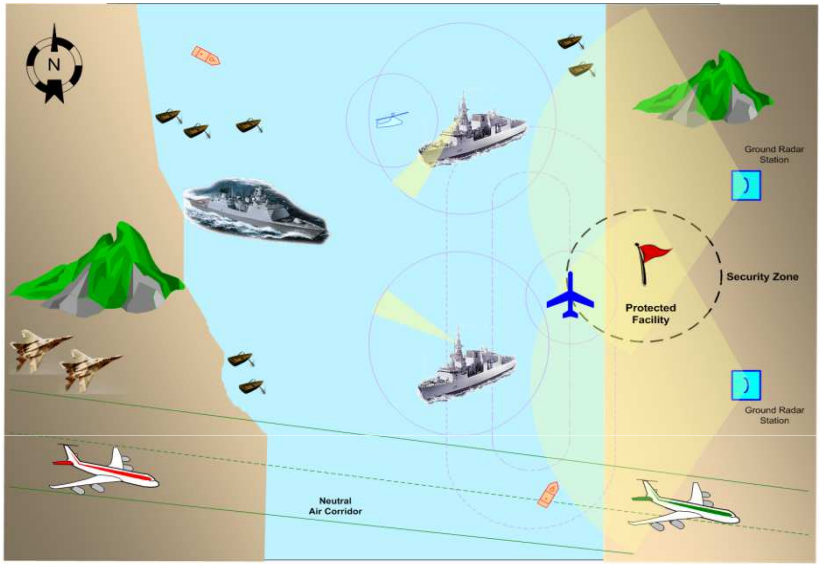
Communication Relay

Outline

- Introduction to Optimization
- **Optimization and Operations Research**
- Optimization Problem
- Optimization Problem Classifications
- Optimization Methods
- Combinatorics
- Computational Complexity [For Reading]

Optimization and Operations Research

- **Operations Research**
  - ◇ Originated in the beginning of **World War II** due to the urgent assignment of scarce resources in military operations, in tactical and strategic problems.



  - ◇ OR has then expanded into a field widely used in industries ranging from petrochemicals to airlines, finance, logistics, and government.

1. Effective Decisions
2. Better Coordination
3. Facilitates Control
4. Improves Productivity

[1]

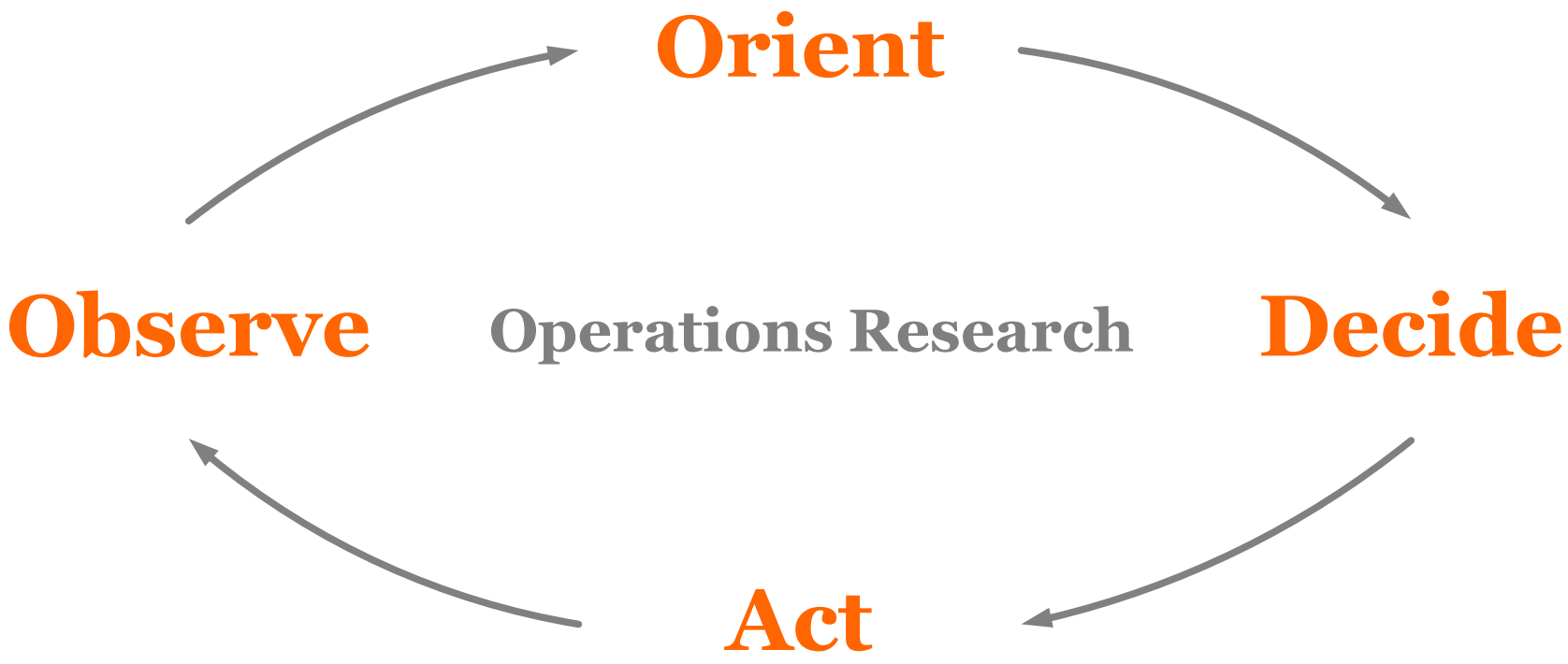
Optimization and Operations Research

- **Operations Research**
  - ◇ Operations research (also referred to as decision science, or management science) is the application of **advanced scientific analytical methods** in **improving the effectiveness** in operations, decisions and management of a company.
    - Design and improvements in **operations** and **decisions**.
    - Problem solving and support in **management, planning or forecasting** functions.
    - Provide knowledge and **decision support**.

[1]

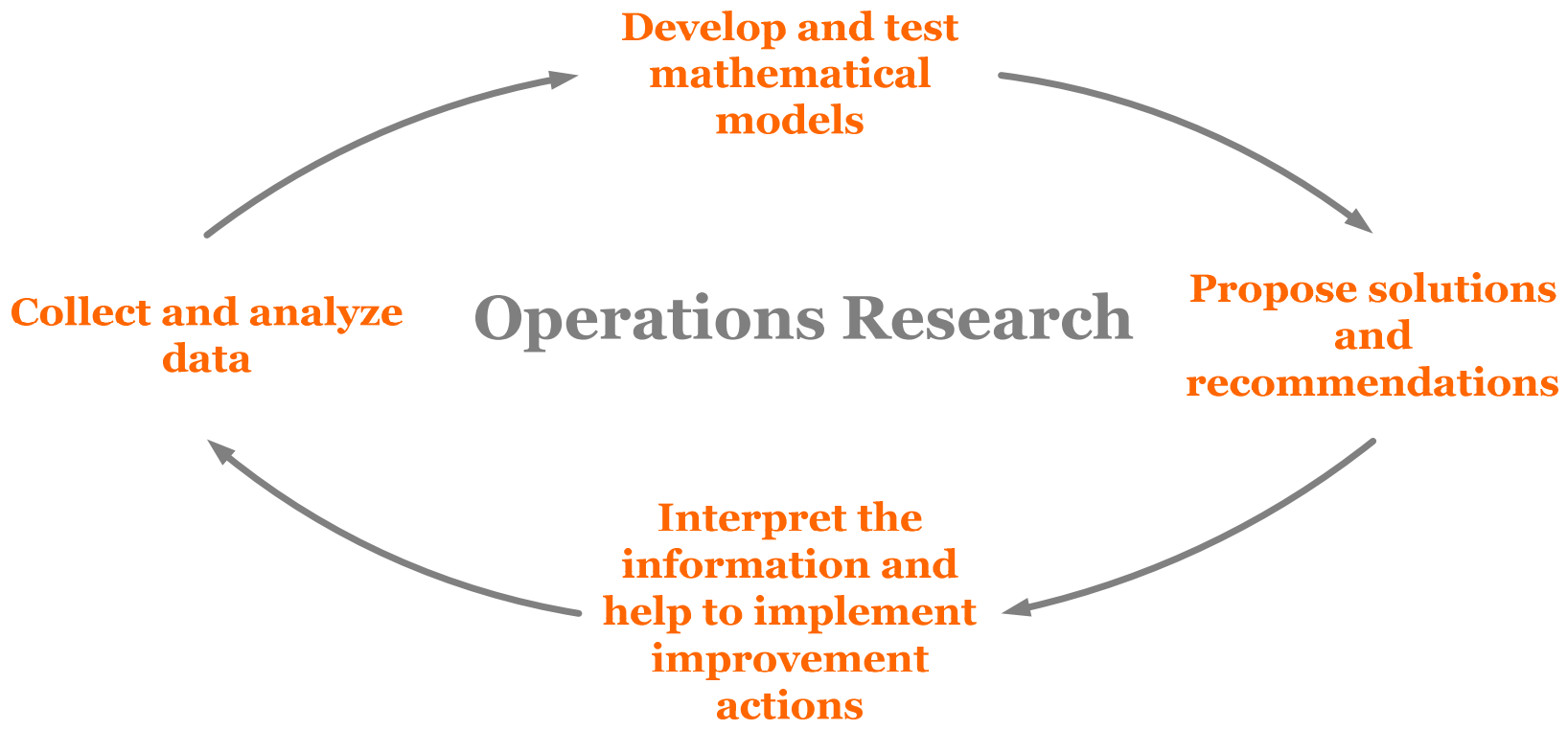
Optimization and Operations Research

- **Operations Research**



Optimization and Operations Research

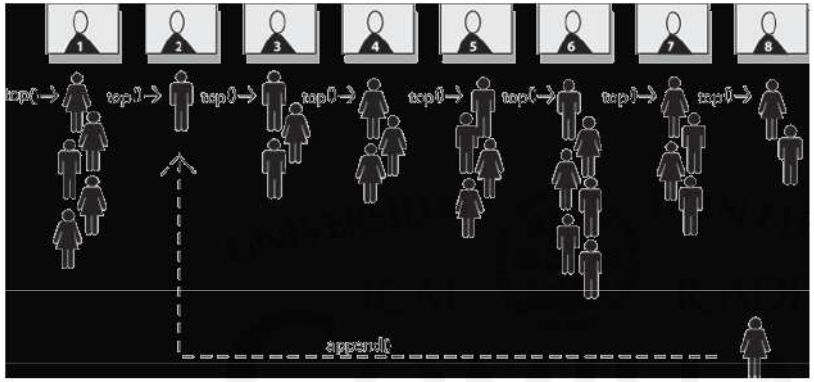
- **Operations Research**



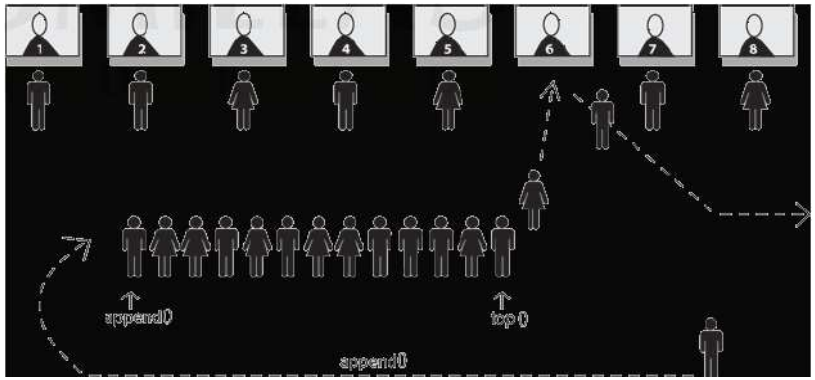
Optimization and Operations Research

- **Operations Research**
  - ◇ Example: Ticket Queue

Ticket Queue



8 Queues – 8 Servers



1 Queue – 8 Servers

[1]

Optimization and Operations Research

- **Optimization in Operations Research**
  - ◇ Operations research is a branch of mathematics concerned with the application of **scientific methods and techniques** to **decision making problems** and with establishing the **best or optimal solutions**.
  - ◇ The **optimum seeking methods** are also known as **optimization techniques** and are generally studied as a **part of operations research**.

[1]

Outline

- Introduction to Optimization
- Optimization and Operations Research
- **Optimization Problem**
- Optimization Problem Classifications
- Optimization Methods
- Combinatorics
- Computational Complexity [For Reading]

Optimization Problem

- **Statement of an Optimization Problem**

  An optimization or a mathematical programming problem can be stated as follows.

$$\text{Find} \quad \mathbf{X} = \begin{Bmatrix} x_1 \\ x_2 \\ . \\ . \\ . \\ x_n \end{Bmatrix} \quad \text{which minimize /maximize} \quad f(\mathbf{X})$$

  subject to the constraints

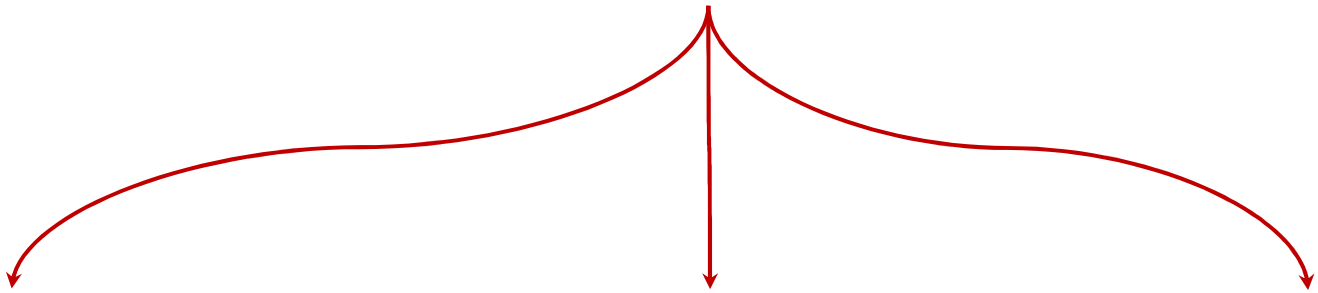
$$g_j(\mathbf{X}) \le 0, \qquad j = 1,2,\ldots,m \qquad \text{Inequality constraints}$$

$$l_j(\mathbf{X}) = 0, \qquad j = 1,2,\ldots,p \qquad \text{Equality constraints}$$

[2]

Optimization Problem

Optimization Problem



Design vector (Set of unknowns or decision variables)

$$\mathbf{X} = \{x_1 \quad x_2 \quad . \quad . \quad . \quad x_n\}^T$$

+

Objective Function

$$f(\mathbf{X})$$

+

Set of inequality and/or equality constraints

$$g_j(\mathbf{X}),\ l_j(\mathbf{X})$$

Note: Some optimization problems do not involve any constraints. Such problems are called unconstrained optimization problems.

Optimization Problem

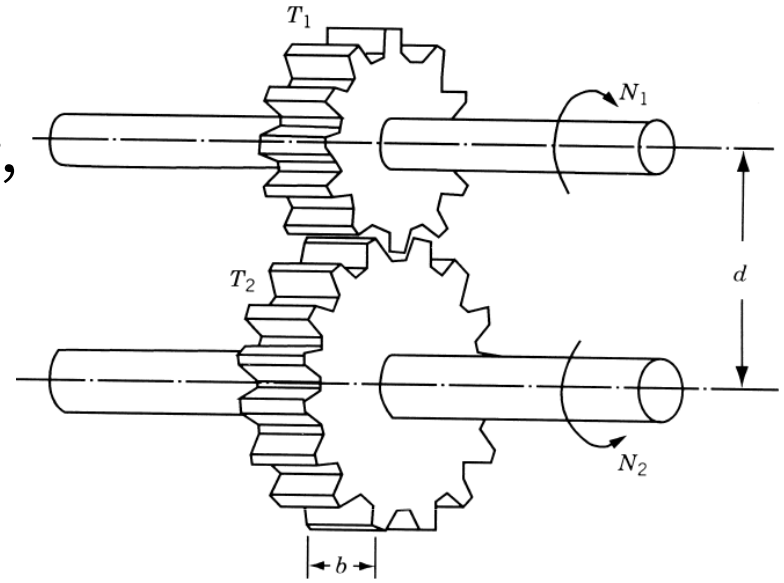
- **Design Vector**

  A set of unknowns or decision variables which affects the value of the objective function.

  If **x** represents the unknowns, also referred to as the independent variables, then ***f*(x) quantifies the quality** of the **candidate solution** or **feasible solution**, **x**.

Optimization Problem

- **Design Vector: Example**
  - ◇ Consider the design of the gear pair, characterized by its face width b, number of teeth T_1 and T_2, center distance d, pressure angle ψ, tooth profile, and material.



  - ◇ If center distance d, pressure angle ψ, tooth profile, and material of the gears are fixed in advance, these quantities can be called **preassigned parameters**.
  - ◇ The remaining quantities can be collectively represented by a **design vector** $\mathbf{X} = \{x_1, x_2, x_3\}^T = \{b, T_1, T_2\}^T$.

[2]

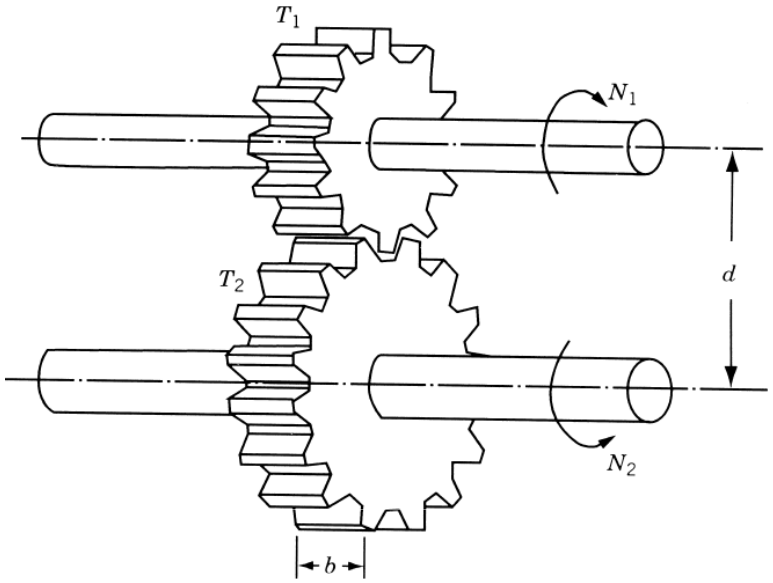
Optimization Problem

- **Design Vector Space**
  - ◇ If there are **no restrictions** on the choice of b, T_1, and T_2, any set of three numbers will constitute a design for the gear pair.
  - ◇ If an n-dimensional Cartesian space with each coordinate axis representing a design variable x_i ($i = 1, 2, \ldots, n$) is considered, the space is called the **design variable space** or simply **design space**.
  - ◇ Each point in the n-dimensional design space is called a **design point** and represents either a possible or an impossible solution to the design problem.

[2]

Optimization Problem

- **Design Vector Space**
  - ◇ In the case of the design of a gear pair,
  - ◇ The design point **{1.0, 20, 40}^{T}**, for example, represents a **possible solution**,



  - ◇ Whereas the design point **{1.0,–20, 40.5}^{T}** represents an **impossible solution**
  - ◇ since it is not possible to have either a negative value or a fractional value for the number of teeth.

[2]

Optimization Problem

- **Set of constraints**

  A set of constraints that **restricts** the values that can be assigned to the unknowns.

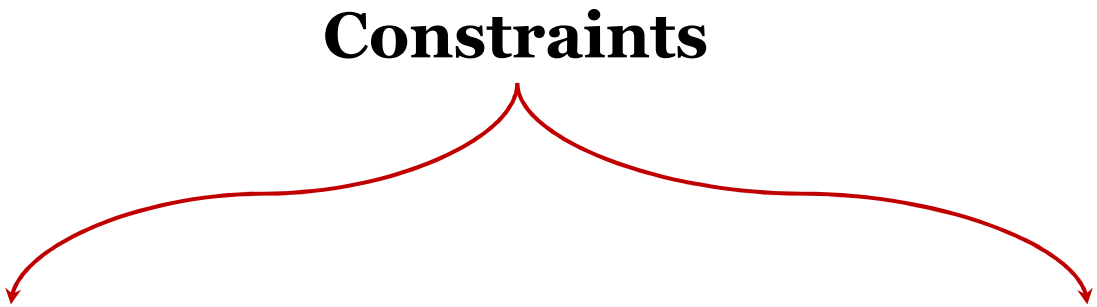
  Most problems define at least a set of **boundary constraints**, which define the domain of values for each variable.

  Constraints can, however, be more **complex, excluding certain candidate solutions** from being considered as solutions.

"*Constrained optimization is the art of compromise between conflicting objectives. This is what design is all about.*" William A. Dembski, Signs of Intelligence: Understanding Intelligent Design

Optimization Problem

- **Set of constraints (cont'd)**



Hard

(must be satisfied)

In your course schedule a **hard constraint** is that no classes overlap

Soft

(is desirable to satisfy)

A **soft constraint** is that no class be before 10 AM

Optimization Problem

- **Set of constraints (cont'd)**
  - ◇ University Course Timetabling Problems: Hard Constraints
    1. **Each teacher** can **only** teach **one class** in the **same timeslot**.
    2. **Each classroom** can only be used for **one course in the same timeslot** (having two courses taught in the same classroom at the same time is not permitted).
    3. **Each class** can only attend **one course** in any given timeslot.
    4. A **three-credit course** must be arranged in consecutive hours, not on two separate days or with an interrupting lunch break.

[3]

Optimization Problem

- **Set of constraints (cont'd)**
  - ◇ University Course Timetabling Problems: Hard Constraints

    5. **No courses** are to be conducted in the third and fourth hours **each Thursday** as that is the time for the weekly **school (class) assembly** and for the class tutor.
    6. According to regulations, teachers holding first-level supervisory positions are to teach **two classes** each week.
    7. Teachers concurrently holding first-level and second-level administrative positions or supervisory positions of academic units **are not to teach any classes on Thursday afternoon**.

[3]

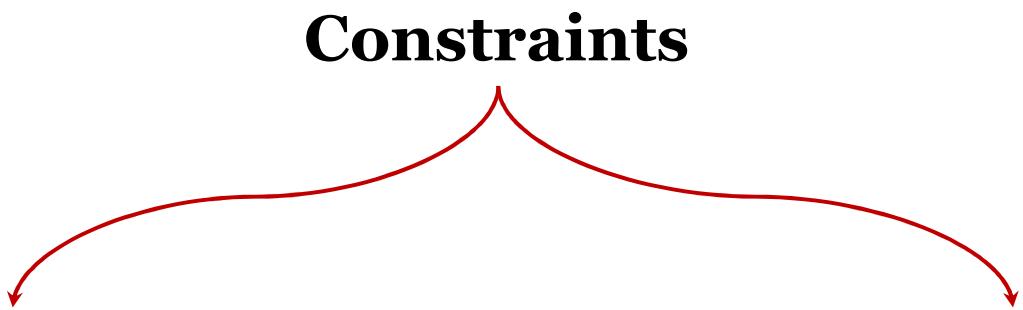
Optimization Problem

- **Set of constraints (cont'd)**
  - ◇ University Course Timetabling Problems: Soft Constraints
    1. **Class time conflicts** between the required courses of each year should be avoided to facilitate students' course retake and credit makeup.
    2. **Course timetabling** should be conducted according to the timeslot **preferences of teachers and students** as well as the levels of preference.
    3. Each teacher may **not teach more hours than the limit** stipulated by the undergraduate department or graduate institute.
    4. Each full-time teacher must teach classes for **at least three days a week**.

[3]

Optimization Problem

- **Set of constraints (cont'd)**



Explicit

(are explicitly stated in a given problem)

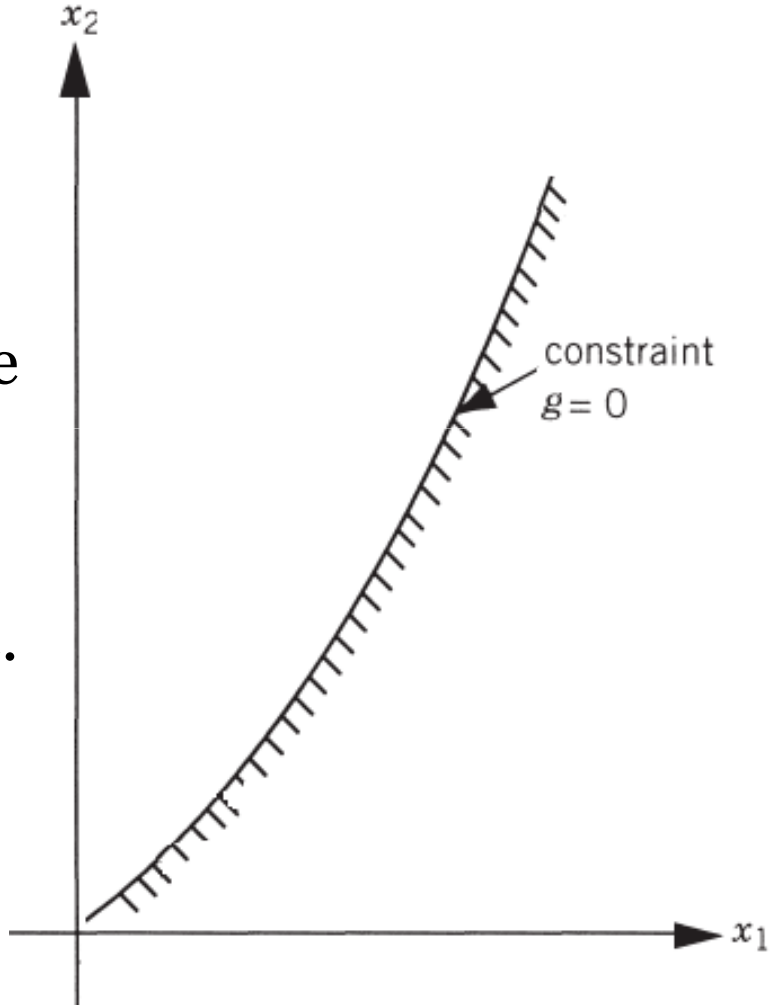
Implicit

(are part of the natural description of the phenomenon under study).

These are typically bound constraints on the decision variables For example, the **width of an object is necessarily non-negative**.

Optimization Problem

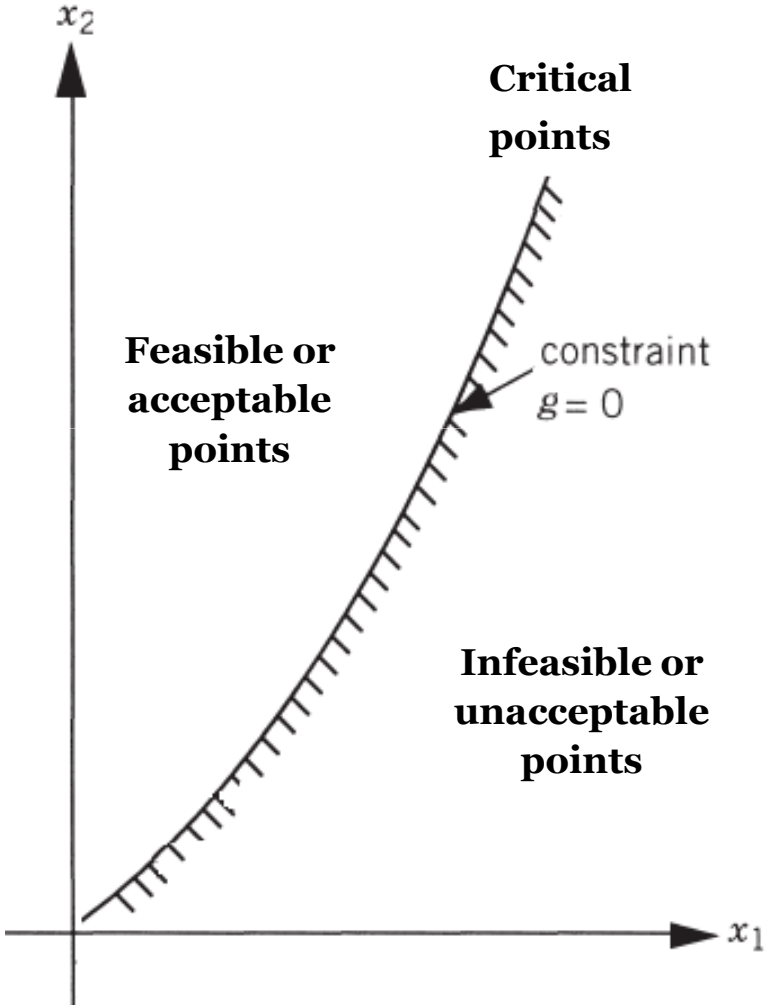
- **Constraint Surface**
  - ◇ consider an optimization problem with only **inequality constraints** $g_j(X) \leq 0$.
  - ◇ The set of values of X that satisfy the equation $g_j(X) = 0$ forms a **hypersurface** in the design space and is called a **constraint surface**.
  - ◇ Note that this is an **(n – 1)-dimensional subspace**, where n is the number of design variables.



[2]

Optimization Problem

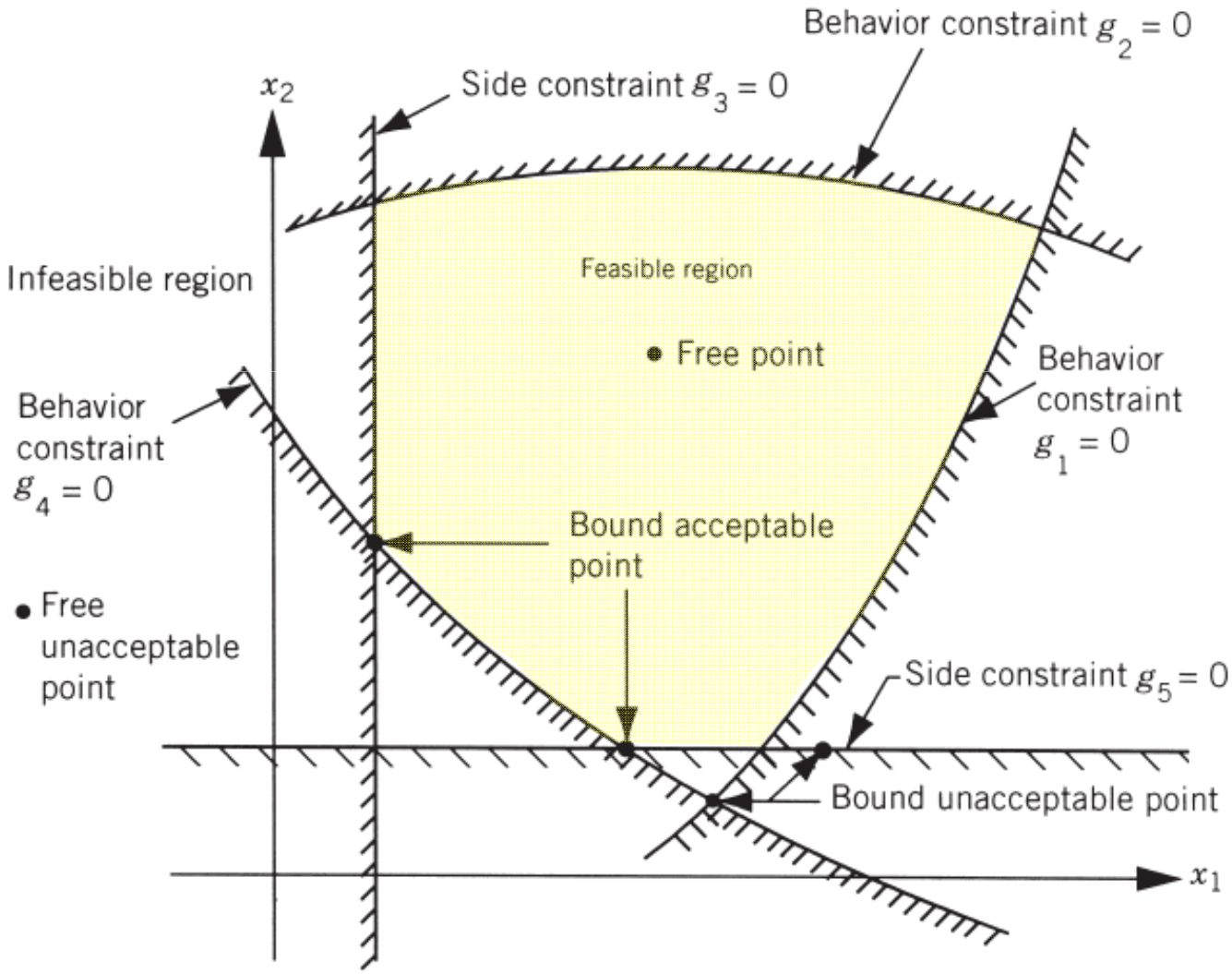
- **Constraint Surface**
  - ◇ The constraint surface divides the design space into two regions:
  - ◇ The points lying on the hypersurface will satisfy the constraint $g_j(X)$ **critically**, whereas the points lying in the region where $g_j(X) > 0$ are **infeasible** or **unacceptable**, and the points lying in the region where $g_j(X) < 0$ are **feasible or acceptable**.



[2]

Optimization Problem

- **Composite Constraint Surface**



4-Types of Points

1. Free and acceptable point
2. Free and unacceptable point
3. Bound and acceptable point
4. Bound and unacceptable point

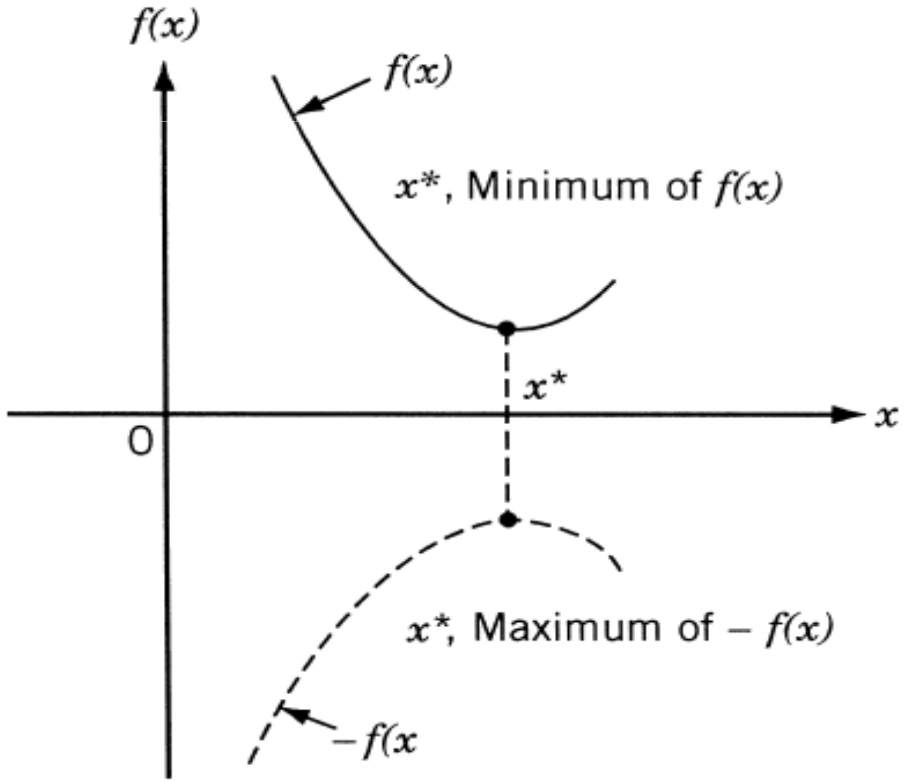
[2]

Optimization Problem

- **Objective Function**

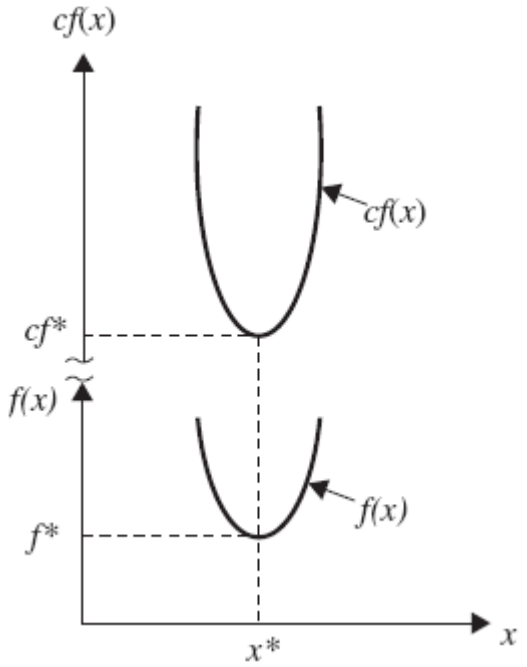
  An objective function (also known as the criterion or merit function) represents the quantity to be optimized, that is, the quantity to be minimized or maximized.

  - ◇ If a point $\boldsymbol{x}^*$ corresponds to the **minimum value** of function $f(x)$,
  - ◇ the same point also corresponds to the **maximum value** of the negative of the function, $-f(x)$.

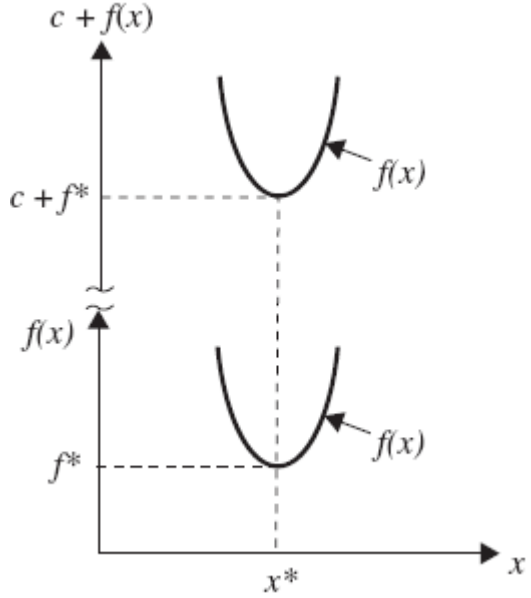


Optimization Problem

- **Objective Function**
  - ◇ The following operations on the objective function will not change the optimum solution x^*.
    - Multiplication (or division) of $f(x)$ by a positive constant c.

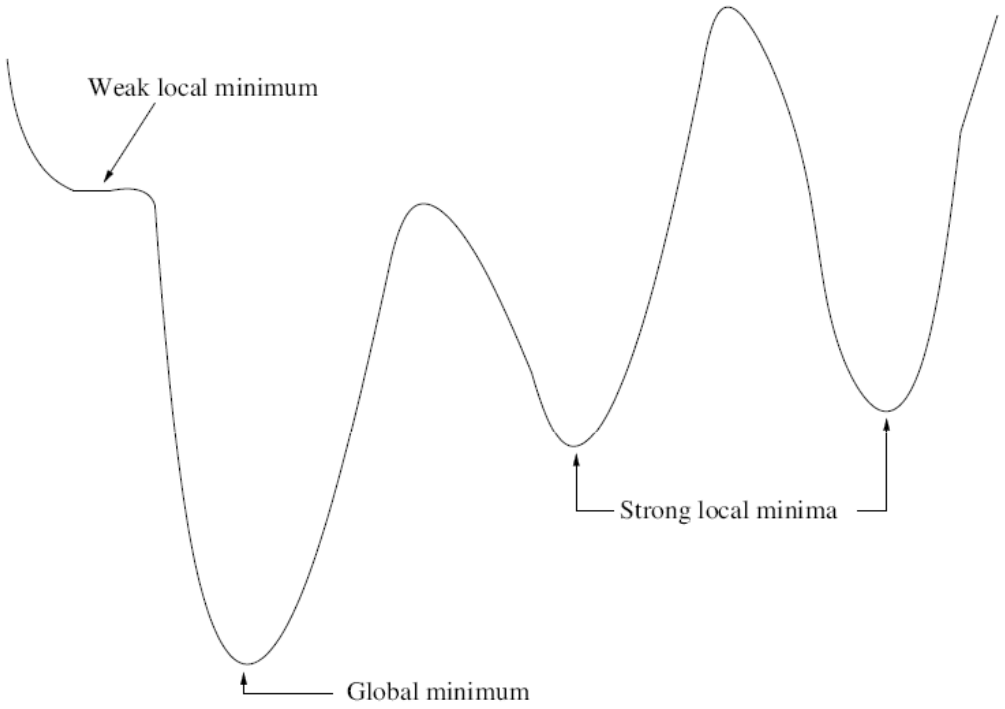


    - Addition (or subtraction) of a positive constant c to (or from) $f(x)$.



Optimization Problem

- **Objective Function: Optimality Conditions**
  - ◇ **Global Optimum:** a global optimum of global minimum (considering a minimization problem) is formally defined as follows:



Global minimum: *The solution* $\mathbf{x}^* \in \mathcal{F}$*, is a global optimum of the objective function,* f*, if*

$$f(\mathbf{x}^*) < f(\mathbf{x}),\ \forall \mathbf{x} \in \mathcal{F}$$

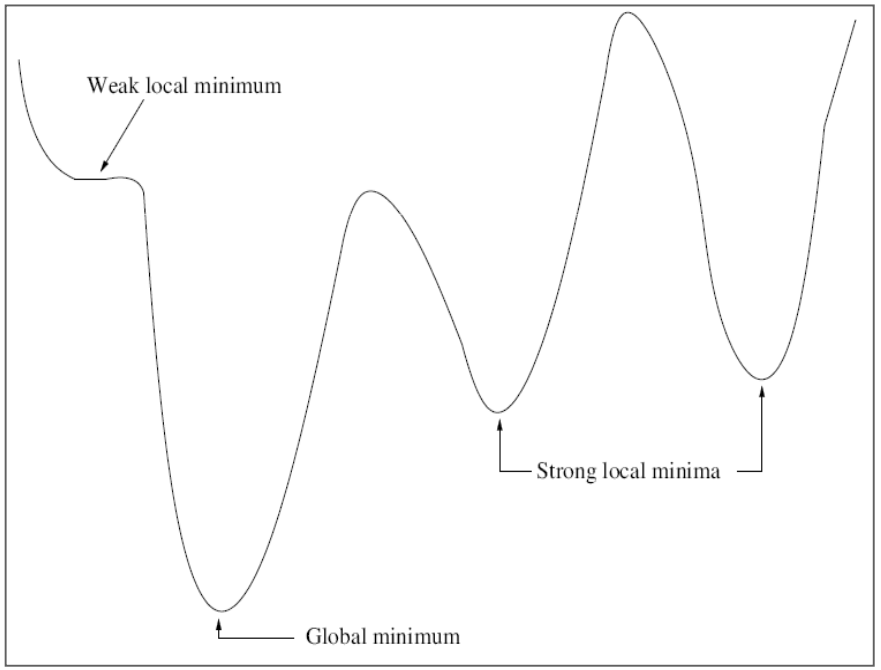
where $\mathcal{F} \subseteq \mathcal{S}$*.*

[4]

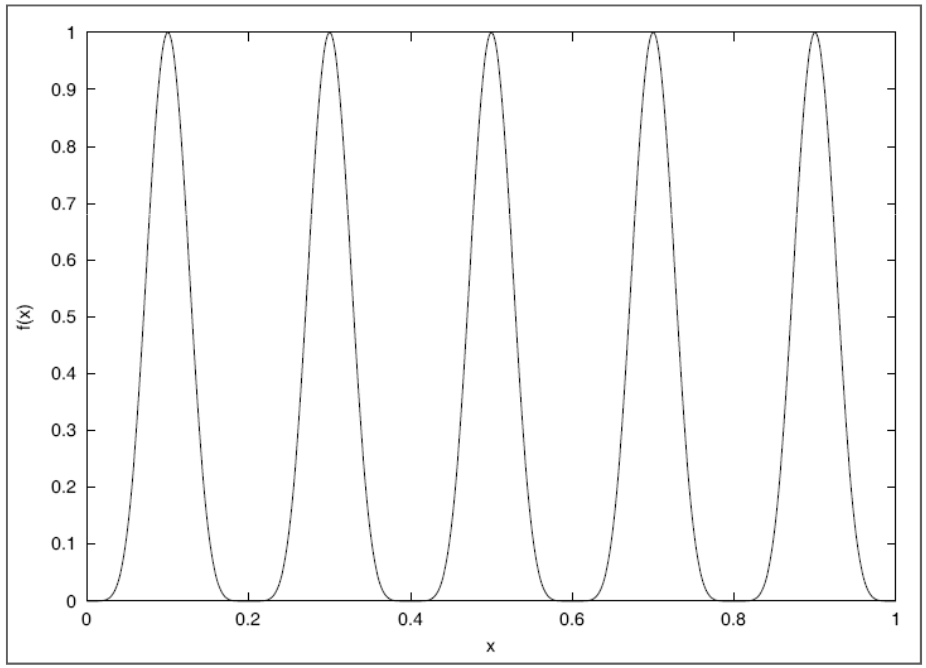
Optimization Problem

- **Objective Function: Optimality Conditions**

  The global optimum is therefore the best of a set of candidate solutions.



Global optimum for a minimization problem

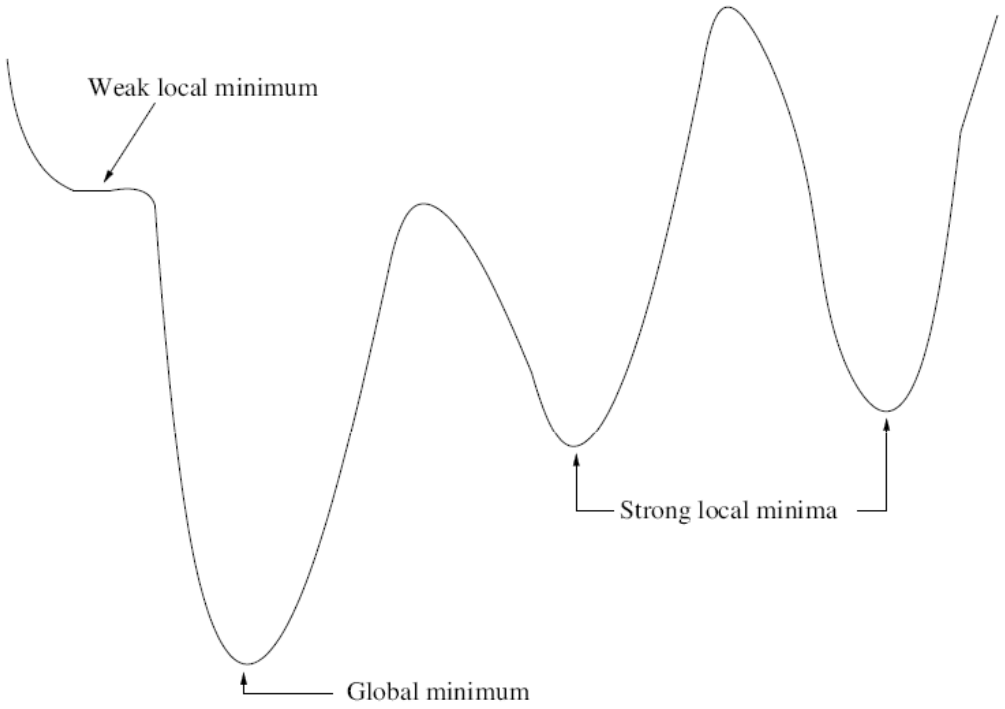


A problem may have more than one global optimum.

$f(x) = \sin^6(5\pi x)$ [4]

Optimization Problem

- **Objective Function: Optimality Conditions**
  - **Strong Local Minimum:** A strong local minimum is defined as follows:



Strong local minimum: *The solution,* $\mathbf{x}^*_{\mathcal{N}} \in \mathcal{N} \subseteq \mathcal{F}$*, is a strong local minimum of* f*, if*

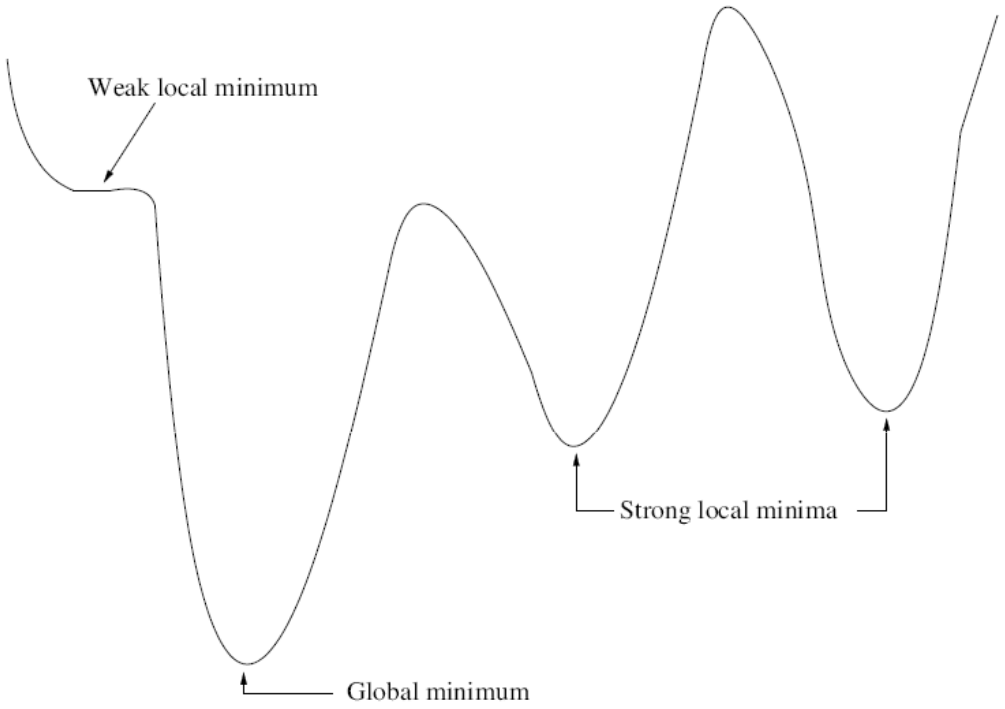
$$f(\mathbf{x}^*_{\mathcal{N}}) < f(\mathbf{x}),\ \forall \mathbf{x} \in \mathcal{N}$$

where $\mathcal{N} \subseteq \mathcal{F}$ *is a set of feasible points in the neighborhood of* $\mathbf{x}^*_{\mathcal{N}}$.

[4]

Optimization Problem

- **Objective Function: Optimality Conditions**
  - **Weak Local Minimum:** A weak local minimum is defined as follows:



Weak local minimum: *The solution,* $\mathbf{x}^*_{\mathcal{N}} \in \mathcal{N} \subseteq \mathcal{F}$*, is a weak local minimum of* f*, if*

$$f(\mathbf{x}^*_{\mathcal{N}}) \leq f(\mathbf{x}), \ \forall \mathbf{x} \in \mathcal{N}$$

where $\mathcal{N} \subseteq \mathcal{F}$ *is a set of feasible points in the neighborhood of* $\mathbf{x}^*_{\mathcal{N}}$.

[4]

Optimization Problem

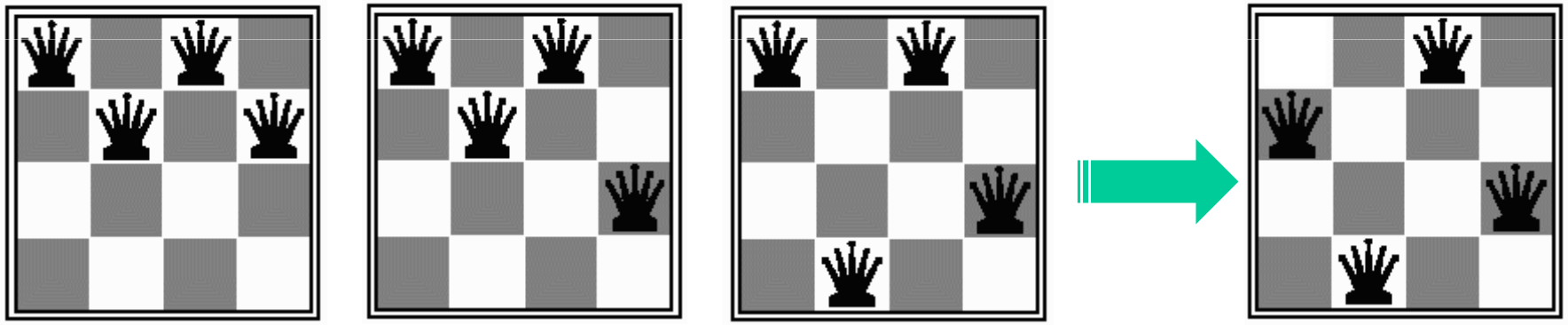
- **Objective Function**

  Some problems, specifically **constraint-satisfaction problems (CSP)**, do not define an explicit objective function. Instead, the objective is to find a solution which satisfies all of a set of constraints.

Optimization Problem

- **Objective Function: Constraint-Satisfaction Problems**
  - ◇ Example: N-queen problem

Put *n* queens on an *n*x*n* board with no two queens on the same row, column, or diagonal



Move a queen to reduce number of conflicts

Goal configuration

no queen is attacking another queen

Optimization Problem

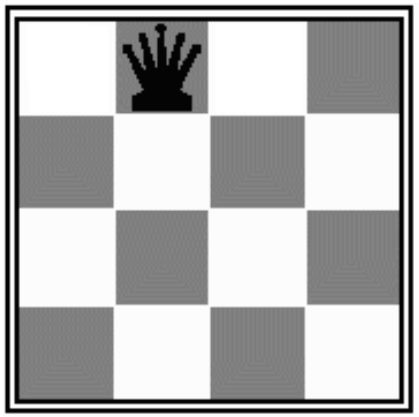
- **Objective Function: Constraint-Satisfaction Problems**
  - ◇ Example: N queen problem

    Assume:

    each queen is represented as a row: Q1, Q2, Q3, and Q4

    An assignment consists of assigning a column to a queen

    { Q1 = 2 } → places a queen in row 1, column 2

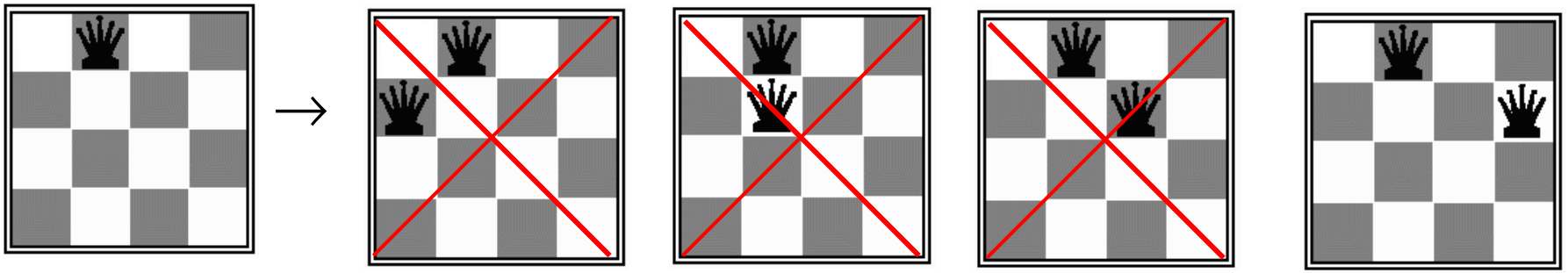


Optimization Problem

- **Objective Function: Constraint-Satisfaction Problems**
  - ◇ Example: N queen problem

    The constraints on the problem restrict certain values for each variable so that all assignments are consistent.

    For example, after we have assigned Q1, and now want to assign Q2, we know we cannot use value 2, since this would violate a constraint: Q1 could attack Q2 and visa versa.



Optimization Problem

- **Objective Function: Constraint-Satisfaction Problems**
  - ◇ Example: N queen problem

Variables: { Q1, Q2, Q3, Q4 }

Domain: { (1, 2, 3, 4), (1, 2, 3, 4), (1, 2, 3, 4), (1, 2, 3, 4) }

Constraints: *Alldifferent*(Q1, Q2, Q3, Q4) and

for i = 0…n and j = (i+1)…n, k = j-i,

Q[i] != Q[j] + k and

Q[i] != Q[j] - k.

[5]

Optimization Problem

- **Objective Function**

  An optimization problem involving **multiple objective functions** is known as a **multiobjective optimization problem**.

  With multiple objectives there arises a **possibility of conflict**, and one simple way to handle the problem is to construct an **overall objective function** as a linear combination of the conflicting multiple objective functions.

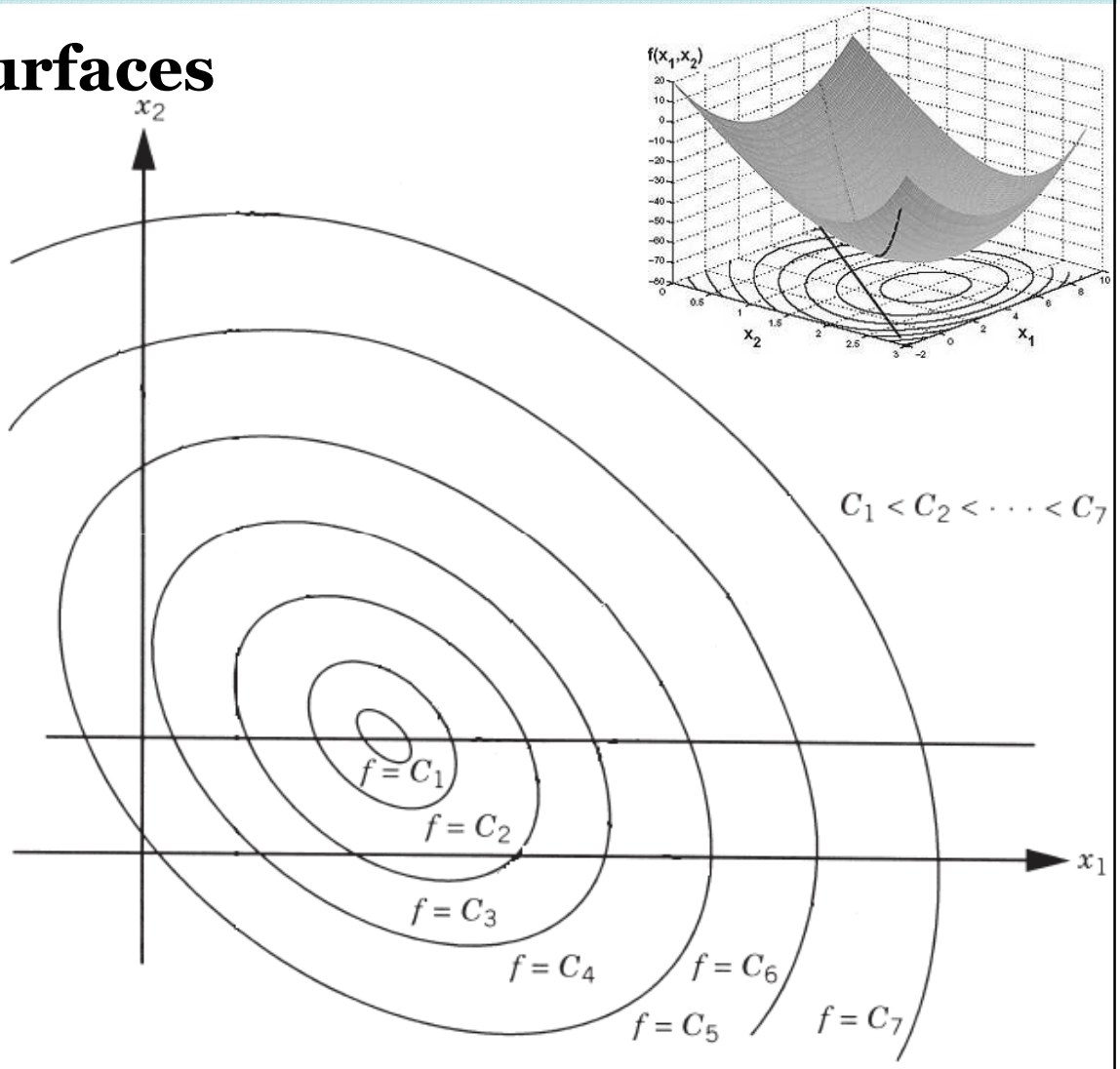
$$f(\mathbf{X}) = \alpha_1 f_1(\mathbf{X}) + \alpha_2 f_2(\mathbf{X})$$

  where α_1 and α_2 are constants whose values indicate the relative importance of one objective function relative to the other.

[2]

Optimization Problem

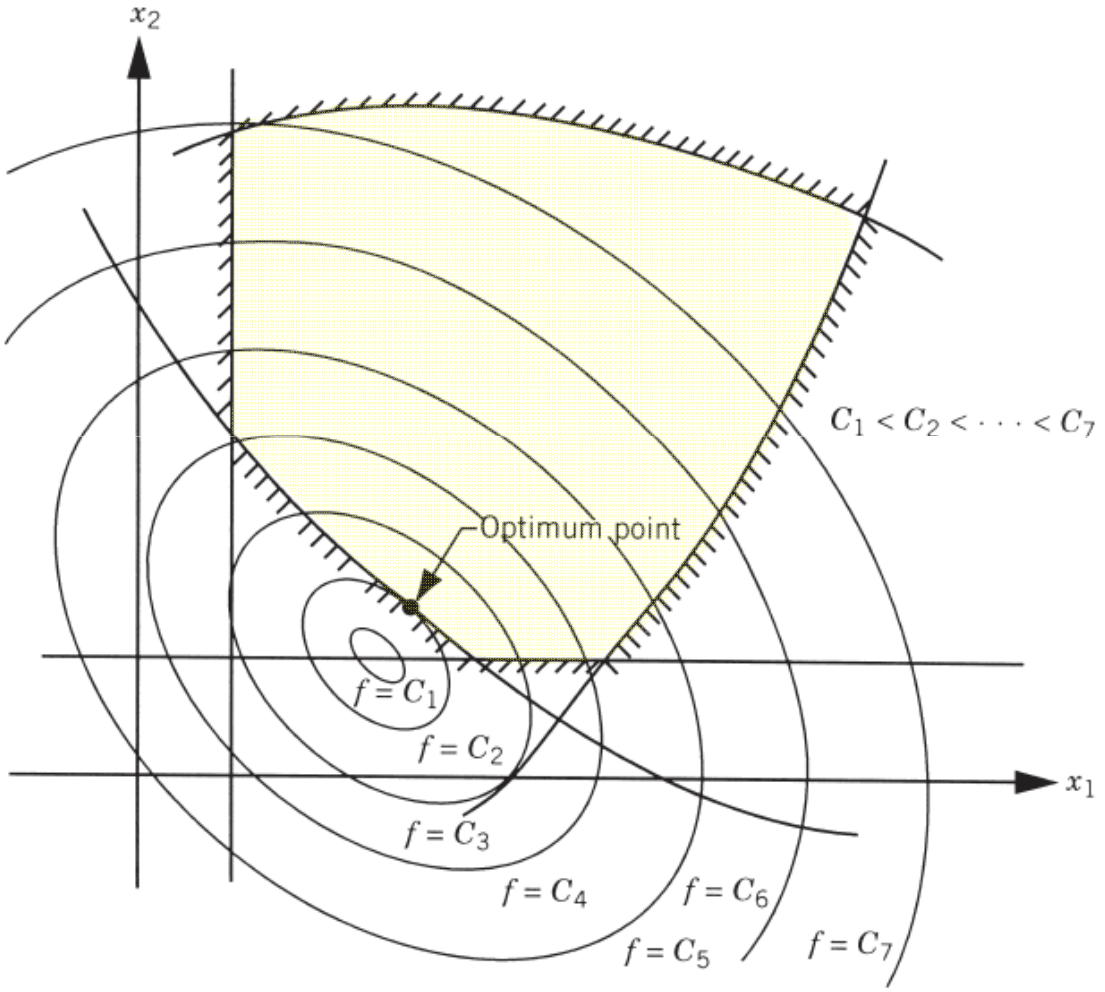
- **Objective Function Surfaces**
  - ◇ The locus of all points satisfying $f(\mathbf{X}) = \mathbf{C}$ = constant forms a **hypersurface** in the design space, and each value of C corresponds to a different member of a family of surfaces.



Objective function surfaces in a hypothetical two-dimensional design space [2]

Optimization Problem

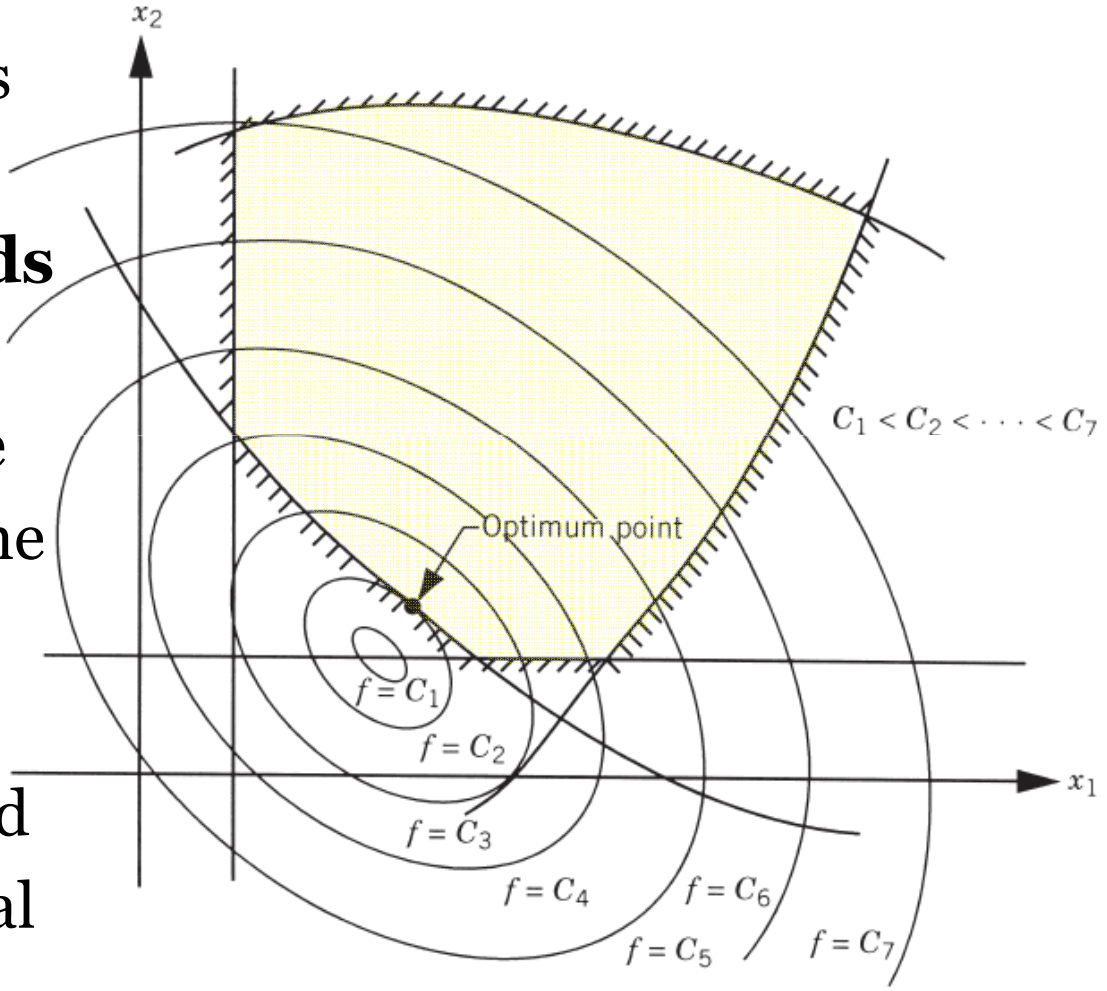
- **Objective Function Surfaces**
  - ◇ Once the **objective function surfaces** are drawn along with the constraint surfaces,
  - ◇ the **optimum point** can be determined without much difficulty.



Objective function surfaces in a hypothetical two-dimensional design space [2]

Optimization Problem

- **Objective Function Surfaces**
  - ◇ But the main problem is that as the number of design variables **exceeds two or three**, the constraint and objective function surfaces become **complex even for visualization** and the problem has to be solved purely as a mathematical problem.



Objective function surfaces in a hypothetical two-dimensional design space [2]

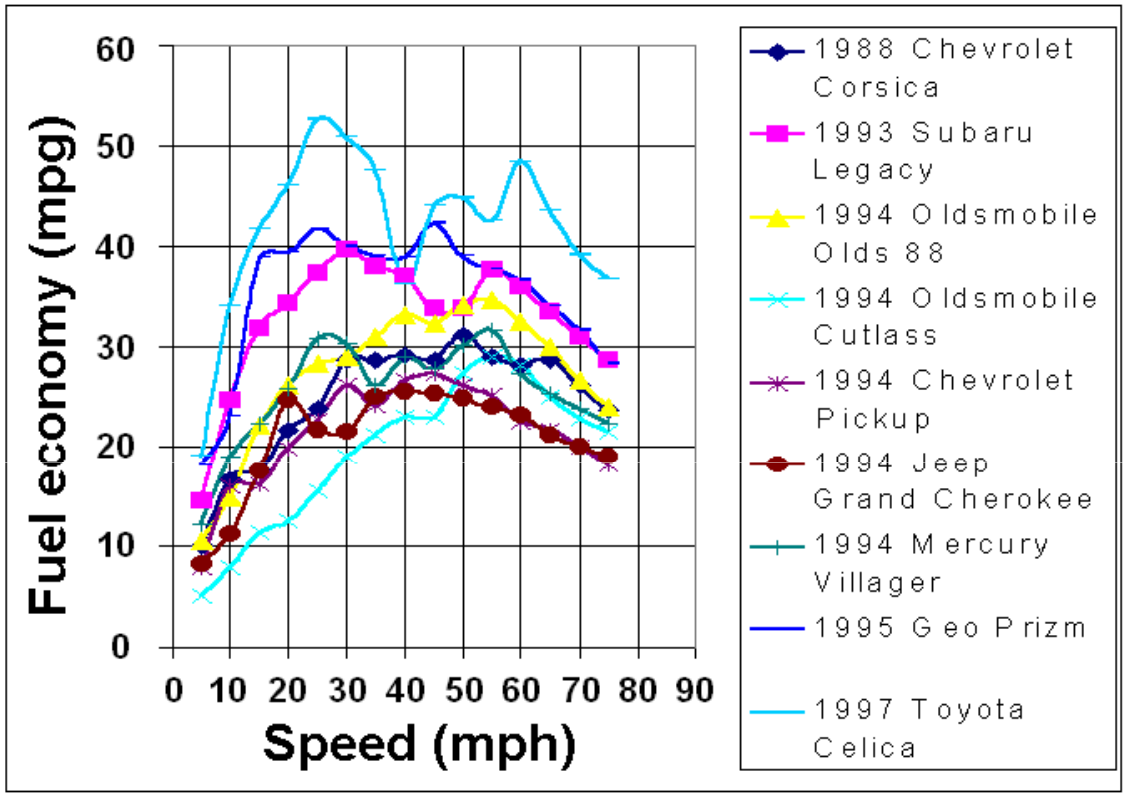
Optimization Problem

- ***Example-1:***

◇ **objective function**

f=fuel economy

distance traveled per unit of fuel used; in miles per gallon (mpg) or kilometres per litre (km/L),



◇ **Design vector or unknown**

$$\mathbf{X} = \{x_1\} = \{\text{car speed in mph or Km/h}\}$$

◇ **Constraints**

$$g_1(\mathbf{X}) = x_1 \leq \text{speed limit}$$

Optimization Problem

- ***Example-2:***

$$\mathbf{X} = \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} \qquad f(\mathbf{X}) = x_1 + x_2$$

$$g_1(\mathbf{X}) = 3x_1 - x_2 \le 3$$
$$g_2(\mathbf{X}) = x_1 + 2x_2 \le 5$$
$$g_3(\mathbf{X}) = x_1 + x_2 \le 4$$
$$g_4(\mathbf{X}) = x_1 \ge 0, x_2 \ge 0$$

$$\text{find} \quad \mathbf{X}^* = \arg\min_{\mathbf{x}} f(\mathbf{X})$$

$$\text{s.t.} \quad g_i(\mathbf{X}) \qquad i = 1,2,3,4$$

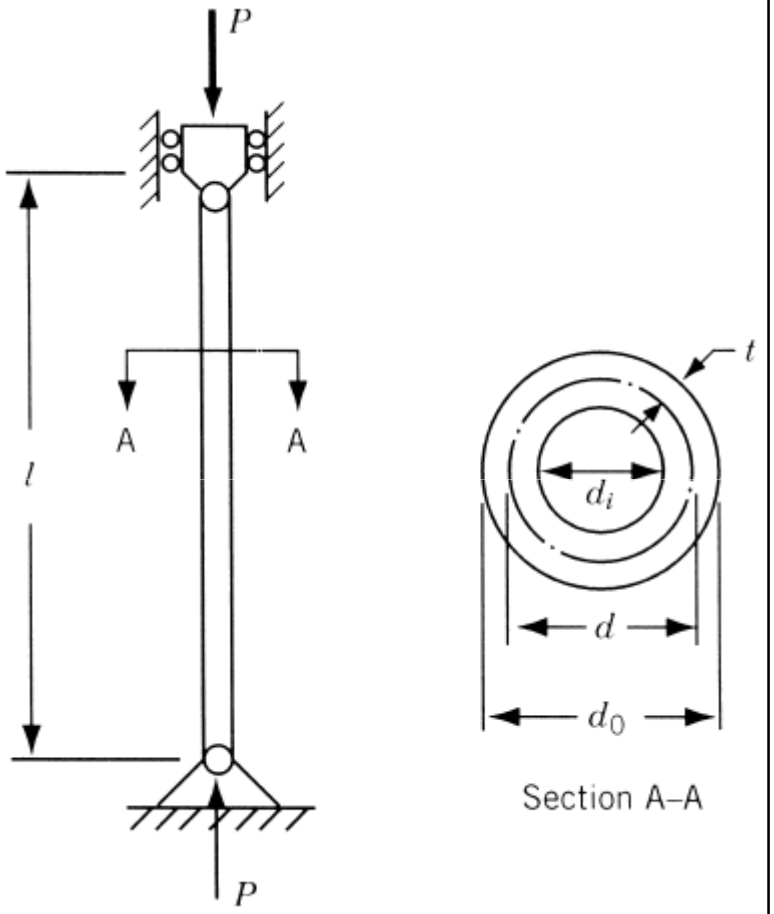
Solution: TBD on the blackboard…

Optimization Problem

- ***Example-3:***

  Design a **uniform column of tubular section**, with hinge joints at both ends, to carry a compressive **load P=2500 kg_f** for minimum cost. The column is made up of a material that has a **yield stress** (σ_y) of **500 kg_f/cm^2**, **modulus of elasticity (*E*) of 0.85 × 10^6 kg_f/cm^2**, and **weight density (ρ) of 0.0025 kg_f/cm^3**.

  The **length** of the column is **250 cm**.



[2]

For Reading

Optimization Problem

- ***Example-3:***

  The **stress induced** in the column should be less than the **buckling stress** as well as the **yield stress**.

  The **mean diameter** of the column is restricted to lie between **2 and 14 cm**, and

  columns with **thicknesses** outside the range **0.2 to 0.8 cm** are not available in the market.

  The **cost** of the column includes material and construction costs and can be taken as **5W + 2d**,

  where **W** is the **weight** in kilograms force and **d** is the mean **diameter** of the column in centimeters.

[2]

For Reading

Optimization Problem

- ***Example-3:***

  The **design variables** are the mean diameter (d) and tube thickness (t):

$$\mathbf{X} = \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \begin{Bmatrix} d \\ t \end{Bmatrix}$$

  The **objective function** to be minimized is given by:

$$f(\mathbf{X}) = 5W + 2d = 5\rho l \pi dt + 2d = 9.82 x_1 x_2 + 2x_1$$

  The **behavior constraints** can be expressed as:

  stress induced ≤ yield stress

  stress induced ≤ buckling stress

For Reading

Optimization Problem

- ***Example-3:***

The **induced stress** is given by:

$$\text{induced stress} = \sigma_i = \frac{P}{\pi dt} = \frac{2500}{\pi x_1 x_2}$$

The **buckling stress** for a pin-connected column is given by:

$$\text{buckling stress} = \sigma_b = \frac{\text{Euler buckling load}}{\text{corss - sectional area}} = \frac{\pi^2 EI}{l^2}\frac{1}{\pi dt}$$

where

I = second moment of area of the cross section of the column

$$I = \frac{\pi}{64}\left(d_0{}^4 - d_i{}^4\right) = \frac{\pi}{8} x_1 x_2 (x_1{}^2 + x_2{}^2)$$

For Reading

Optimization Problem

- ***Example-3:***

Thus the behavior constraints can be restated as:

$$g_1(\mathbf{X}) = \frac{2500}{\pi x_1 x_2} - 500 \le 0$$

$$g_2(\mathbf{X}) = \frac{2500}{\pi x_1 x_2} - \frac{\pi^2 (0.85\times 10^6)(x_1^2 + x_2^2)}{8(250)^2} \le 0$$

The side/bound constraints are given by

$$2 \le d \le 14$$

$$0.2 \le t \le 0.8$$

which can be expressed in standard form as

$$g_3(\mathbf{X}) = -x_1 + 2.0 \le 0$$

$$g_4(\mathbf{X}) = x_1 - 14.0 \le 0$$

$$g_5(\mathbf{X}) = -x_2 + 0.2 \le 0$$

$$g_6(\mathbf{X}) = x_2 - 0.8 \le 0$$

For Reading

Optimization Problem

- ***Example-3:***

  Problem Statement

Find $\mathbf{x} = \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \begin{Bmatrix} d \\ t \end{Bmatrix}$ which minimize $f(\mathbf{X}) = 5W + 2d = 9.82x_1x_2 + 2x_1$

subject to $g_1(\mathbf{X}) = \dfrac{2500}{\pi x_1 x_2} - 500 \le 0$

$$g_2(\mathbf{X}) = \frac{2500}{\pi x_1 x_2} - \frac{\pi^2 (0.85\times 10^6)(x_1^2 + x_2^2)}{8(250)^2} \le 0$$

$$g_3(\mathbf{X}) = -x_1 + 2.0 \le 0$$

$$g_4(\mathbf{X}) = x_1 - 14.0 \le 0$$

$$g_5(\mathbf{X}) = -x_2 + 0.2 \le 0$$

$$g_6(\mathbf{X}) = x_2 - 0.8 \le 0$$

Since there are only two design variables, the problem **can be solved graphically**.

For Reading

Optimization Problem

- ***Example-3:***
  - ◇ Constraint Surfaces

$$g_1(\mathbf{X}) = \frac{2500}{\pi x_1 x_2} - 500 \le 0 \quad \Rightarrow x_1 x_2 \ge 1.593$$

x_1	2.0	4.0	6.0	8.0	10.0	12.0	14.0
x_2	0.7965	0.3983	0.2655	0.1990	0.1593	0.1328	0.1140

Curve

$$g_2(\mathbf{X}) = \frac{2500}{\pi x_1 x_2} - \frac{\pi^2 (0.85\times 10^6)(x_1^2 + x_2^2)}{8(250)^2} \le 0 \quad \Rightarrow x_1 x_2 (x_1^2 + x_2^2) \ge 47.3$$

x_1	2	4	6	8	10	12	14
x_2	2.41	0.716	0.219	0.0926	0.0473	0.0274	0.0172

Curve

$$g_3(\mathbf{X}) = -x_1 + 2.0 \le 0$$
$$g_4(\mathbf{X}) = x_1 - 14.0 \le 0$$
$$g_5(\mathbf{X}) = -x_2 + 0.2 \le 0$$
$$g_6(\mathbf{X}) = x_2 - 0.8 \le 0$$

Side/bound constraints represent straight lines.

For Reading

Optimization Problem

- ***Example-3:***
  - ◇ Objective Function Surfaces

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = c = \text{constant}$$

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = 50.0$$

x_2	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
x_1	16.77	12.62	10.10	8.44	7.24	6.33	5.64	5.07

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = 40.0$$

x_2	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
x_1	13.40	10.10	8.08	6.75	5.79	5.06	4.51	4.05

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = 31.58$$

x_2	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
x_1	10.57	7.96	6.38	5.33	4.57	4.00	3.56	3.20

For Reading

Optimization Problem

- ***Example-3:***
  - ◇ Objective Function Surfaces

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = c = \text{constant}$$

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = 26.53$$

x_2	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
x_1	8.88	6.69	5.36	4.48	3.84	3.36	2.99	2.69

$$f(\mathbf{X}) = 9.82x_1x_2 + 2x_1 = 20.0$$

x_2	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
x_1	6.70	5.05	4.04	3.38	2.90	2.53	2.26	2.02

For Reading

Optimization Problem

- ***Example-3:***
  - ◇ Graphical Optimization

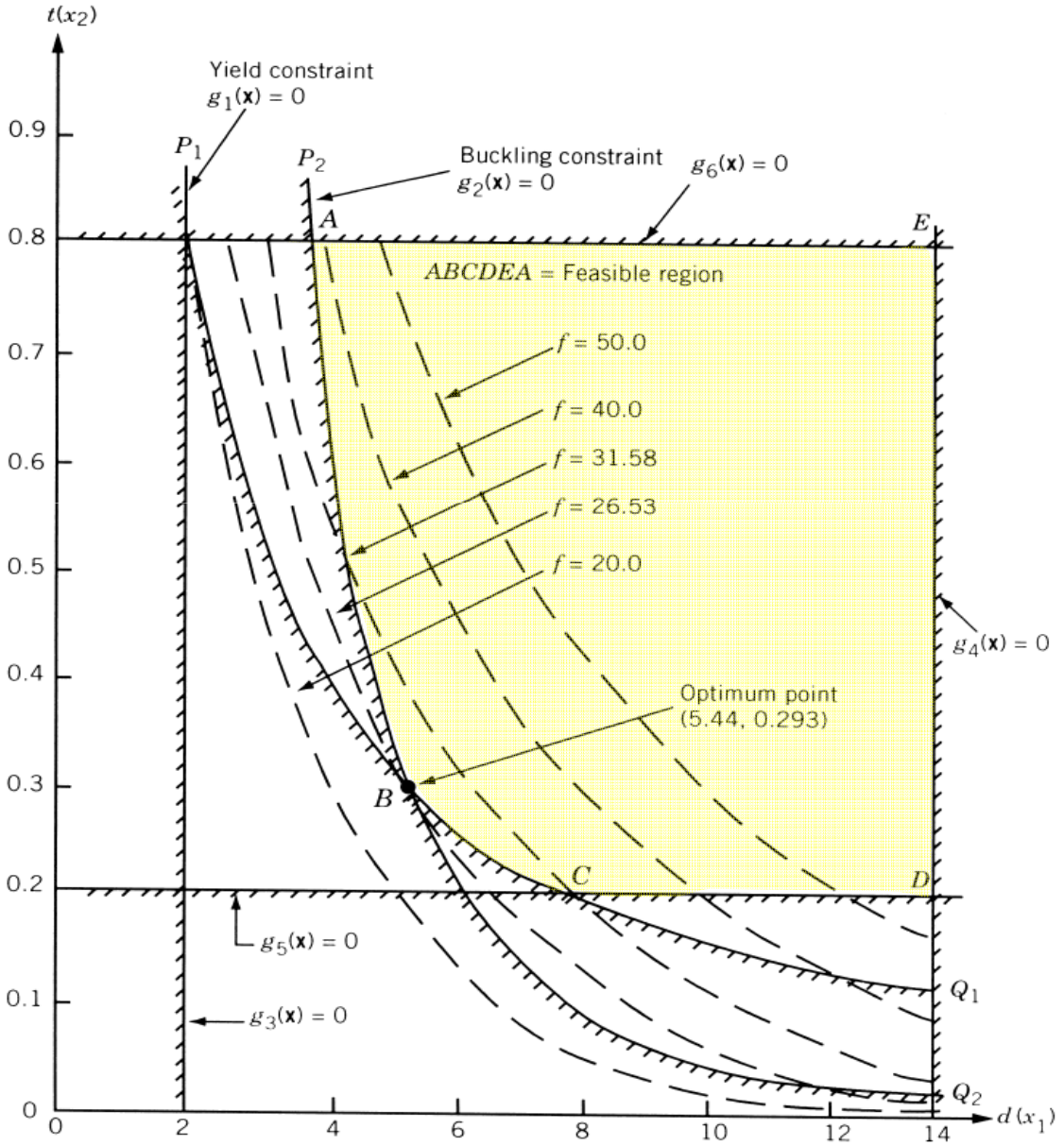
It can be seen that the objective function cannot be reduced below a value of 26.53 (corresponding to point B) without violating some of the constraints.

Thus the optimum solution is given by point B with

$\boldsymbol{d^* = x_1^* = 5.44\ cm}$ and

$\boldsymbol{t^* = x_2^* = 0.293\ cm}$ with

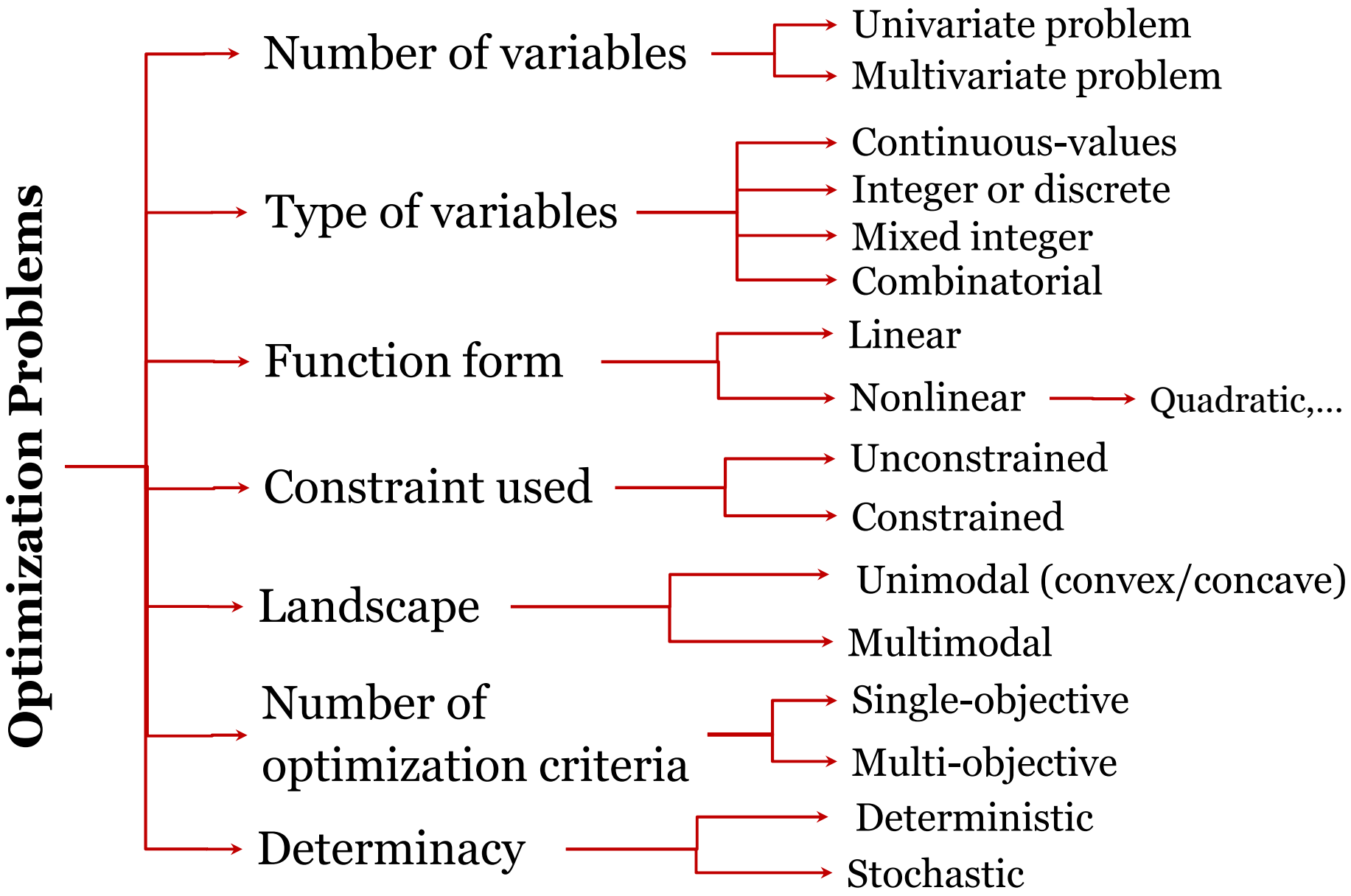
$\boldsymbol{f_{min} = 26.53}$.



Outline

- Introduction to Optimization
- Optimization and Operations Research
- Optimization Problem
- **Optimization Problem Classifications**
- Optimization Methods
- Combinatorics
- Computational Complexity [For Reading]

Optimization Problem Classifications



Optimization Problem Classifications

- **Number of variables**

  The number of variables that influences the objective function:

  A problem with **only one variable** to be optimized is referred to as a **univariate problem**.

  If more than one variable is considered, the problem is referred to as a **multivariate problem**.

Optimization Problem Classifications

- **The type of variables**

  A **continuous problem** has continuous-valued variables,

  i.e. $x_j \in \mathbb{R}, \text{ for each } j = 1, \ldots, n_x$

  If $x_i \in \mathbb{Z}$, the problem is referred to as an **integer or discrete optimization** problem.

  A **mixed integer problem** has both continuous-valued and integer-valued variables.

  Problems where solutions are **permutations of integer-valued** variables are referred to as **combinatorial optimization problems** (finding an optimal object from a finite set of objects).

Optimization Problem Classifications

- **Function form**

  Linear problems have an objective function which is linear in the variables.

  Quadratic problems use quadratic functions.

  When other nonlinear objective functions are used, the problem is referred to as a **nonlinear problem**.

Optimization Problem Classifications

- **The constraints used**

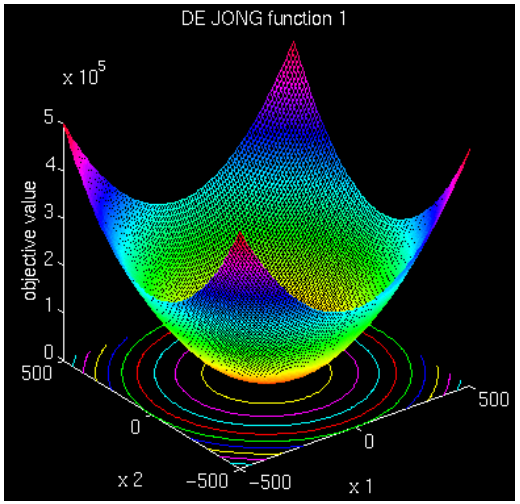
  A problem which uses only **boundary constraints** is referred to as an **unconstrained problem**.

  Constrained problems have additional **equality and/or inequality constraints**.

Optimization Problem Classifications

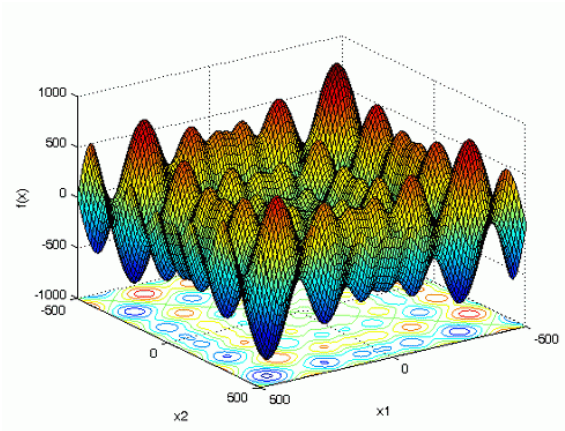
- **Landscape**

  If there exists only one clear solution, the problem is **unimodal**.



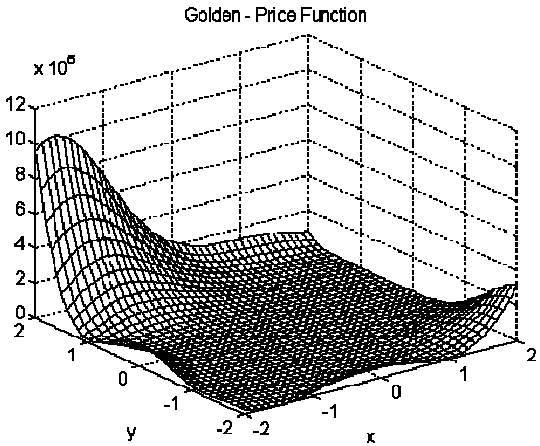
Unimodal or Convex function

If more than one optimum exists, the problem is **multimodal**.



Multimodal function

Some problems may have false optima, in which case the problem is referred **deceptive**.



Deceptive function

Optimization Problem Classifications

- **The number of optimization criteria**

  If the quantity to be optimized is expressed using **only one objective function**, the problem is referred to as a **mono-objective** or **uni-objective problem**.

  A **multi-objective problem** specifies more than one sub-objective which need to be simultaneously optimized.

Optimization Problem Classifications

- **Determinacy**

  All the optimization problems are **deterministic** in the sense that for any given set of design variables, the values of both **objective functions and the constraint functions are determined exactly**.

  In the **real world**, we can only know some parameters to a certain extent with uncertainty. Thus, if there is any **uncertainty and noise in the design variables and the objective functions and/or constraints**, then the optimization becomes a **stochastic optimization problem**, or a robust optimization problem with noise.

Optimization Problem Classifications

- ***Examples:*** State the type of the following optimization problems:

  a) max $\exp(-x^2)$ subject to $-2 \leq x \leq 5$;

  Constrained nonlinear optimization with a single objective (uni-objective).

Optimization Problem Classifications

- ***Examples (cont'd):***

  b) min $x^2 + y^2, (x-2)^2, (y-3)^3$

  Unconstrained multiobjective optimization.

Optimization Problem Classifications

- ***Examples (cont'd):***

  c) Design a quieter, more efficient, low cost car engine;

  Multiobjective optimization with multiple constraints.

Optimization Problem Classifications

- **Simple vs. hard optimization problems**

Simple Problems	Hard Problems
Few decision variables	Many decision variables
Differentiable	Discontinuous, combinatorial
Unimodal	Multimodal
Objective easy to calculate	Objective difficult to calculate
No or light constraints	Severely constraints
Feasibility easy to determine	Feasibility difficult to determine
Single objective	Multiple objective
Deterministic	Stochastic

Outline

- Introduction to Optimization
- Optimization and Operations Research
- Optimization Problem
- Optimization Problem Classifications
- **Optimization Methods**
- Combinatorics
- Computational Complexity [For Reading]

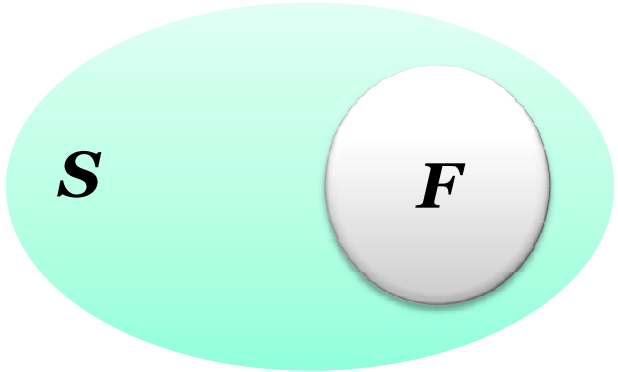
Optimization Methods

- **Optimization Method**

  The goal of an optimization method is then to assign values, from the **allowed domain**, to the **design vector/unknowns** such that the **objective function is optimized and all constraints are satisfied**.

  To achieve this goal, the optimization algorithm **searches for a solution** in a search/design space, ***S***, of candidate solutions.

  In the case of **constrained problems**, a solution is found in the **feasible space**, $F \subseteq S$.



Optimization Methods

- **Optimization Algorithms**

  Optimization techniques are **search methods**, where the goal is to find a solution to an optimization problem, such that a given quantity is optimized, possibly subject to a set of constraints.

  An optimization algorithm **searches for an optimum solution** by **iteratively transforming a current candidate solution into a new**, hopefully better, solution.

Optimization Methods

- **Classifications based on problem characteristics**
  - ◇ **Unconstrained methods:** used to optimize unconstrained problems;
  - ◇ **Constrained methods:** used to find solutions in constrained search spaces;
  - ◇ **Multi-objective optimization methods:** for problems with more than one objective to optimize;
  - ◇ **Multi-solution (niching) methods:** with the ability to locate more than one solution; and
  - ◇ **Dynamic methods:** with the ability to locate and track changing optima.

Optimization Methods

- **Classifications based on type of solution that is located**
  - ◇ **Local Search:** uses only local information of the search space surrounding the current solution to produce a new solution. Since only local information is used, local search algorithms (**local optimizer**) locate **local optima** (which may be a global minimum).
  - ◇ **Global Search:** uses more information about the search space to locate a **global optimum**.
  - ◇ It is said that **global search algorithms** explore the **entire search space**, while **local search algorithms** exploit **neighborhoods**.

Optimization Methods

- **Classifications based on optimization speed**
  - ◇ **Online Optimization:** handle tasks that need to be solved quickly in a time span between **ten milliseconds** to **a few minutes**. In order to find a solution in this short time, optimality is normally **traded in for speed gains**.

    Examples: robot localization, load balancing, services composition for business processes, or updating a factory's machine job schedule after new orders came in.

    From the examples, it becomes clear that online optimization tasks are often carried out **repetitively** – new orders will, for instance, continuously arrive in a production facility and need to be scheduled to machines in a way that minimizes the waiting time of all jobs.

Optimization Methods

- **Classifications based on optimization speed**
  - ◇ **Offline Optimization:** time is not so important and a user is willing to wait maybe even days if he/she can get an optimal or close-to-optimal result.

    Such problems regard for example **design optimization, data mining**, or creating **long-term schedules for transportation crews**.

    These optimization processes will usually be **carried out only once** in a long time.

Optimization Methods

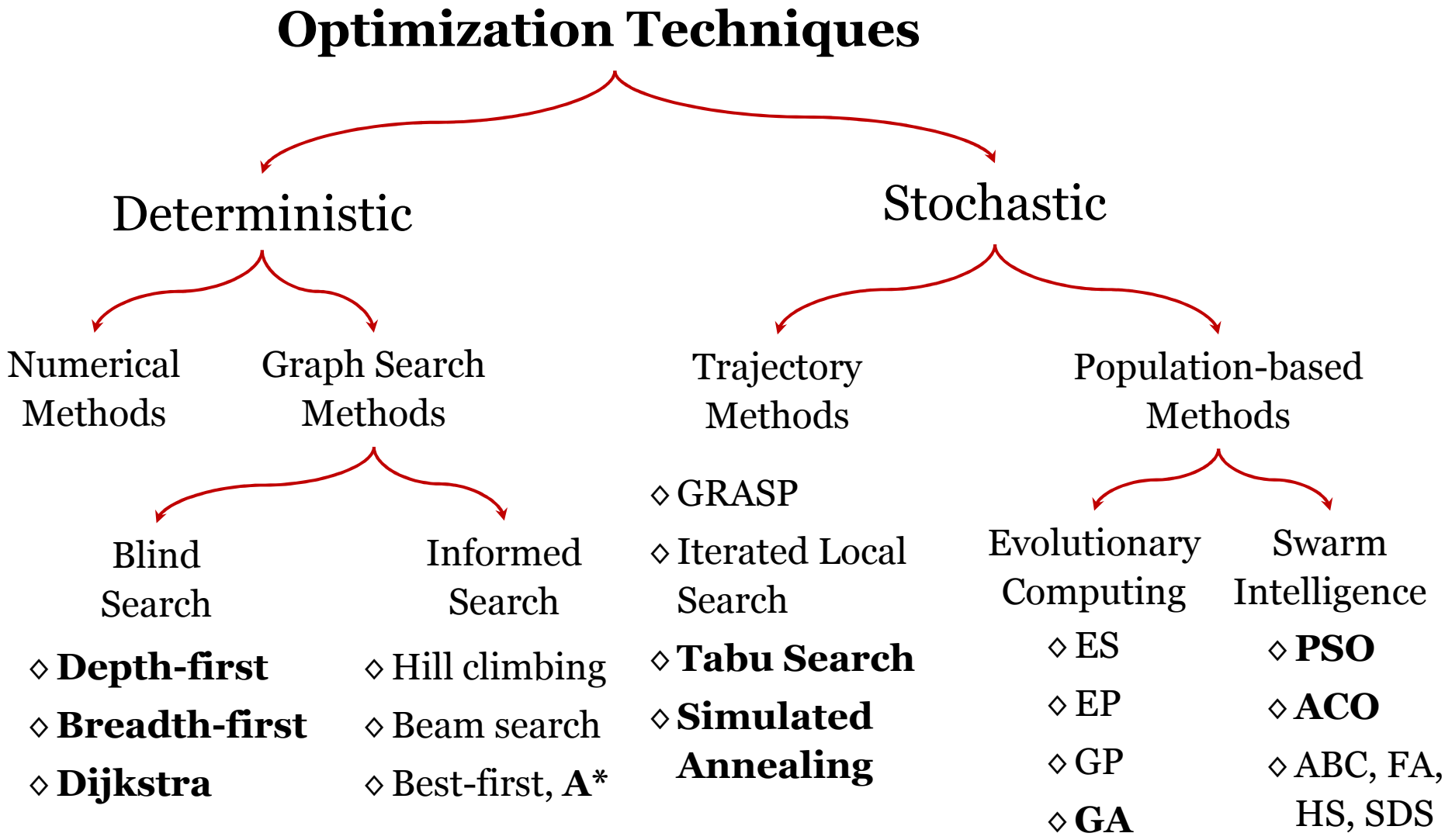
- **Other Classifications**
  - ◇ **Stochastic methods:** use random elements to transform one candidate solution into a new solution. The new point can therefore not be predicted.
  - ◇ **Deterministic methods:** on the other hand, do not make use of random elements.

Optimization Methods

- **Other Classifications**
  - ◇ **Exact Methods**
    - Find the optimal solution,
    - High computational costs.
  - ◇ **Approximate Methods**
    - Find sub-optimal solutions,
    - Low computational costs.

Optimization Methods

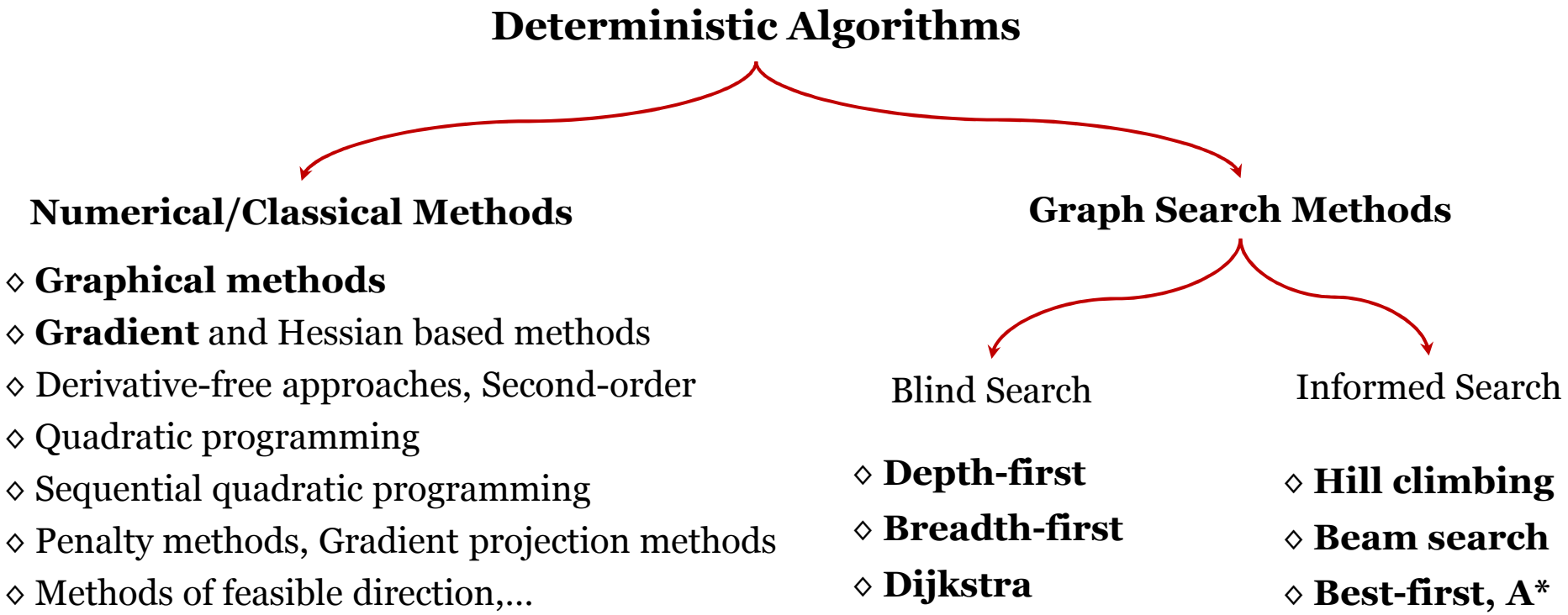
- **Our Classifications**



Optimization Methods

- **Deterministic Algorithms**

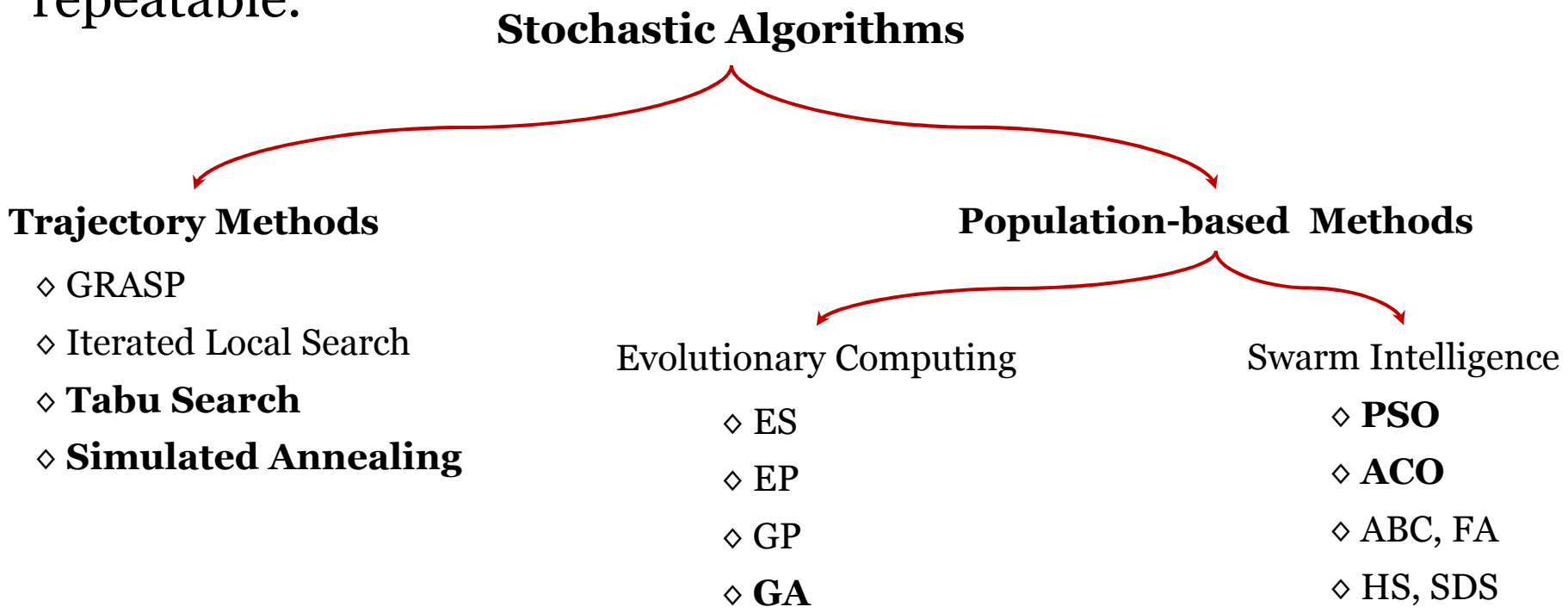
  Deterministic algorithms follow a rigorous procedure and its path and values of both design variables and the functions are repeatable. For the same starting point, they will follow the same path whether you run the program today or tomorrow.



Optimization Methods

- **Stochastic Algorithms**

  Stochastic algorithms always have **some randomness**. For example in genetic algorithm and since this algorithm uses some pseudorandom numbers, though the final results may be no big difference, but the paths of each individual are not exactly repeatable.



Outline

- Introduction to Optimization
- Optimization and Operations Research
- Optimization Problem
- Optimization Problem Classifications
- Optimization Methods
- **Combinatorics**
- Computational Complexity [For Reading]

Combinatorics

Combinations and Permutations: what is the difference?

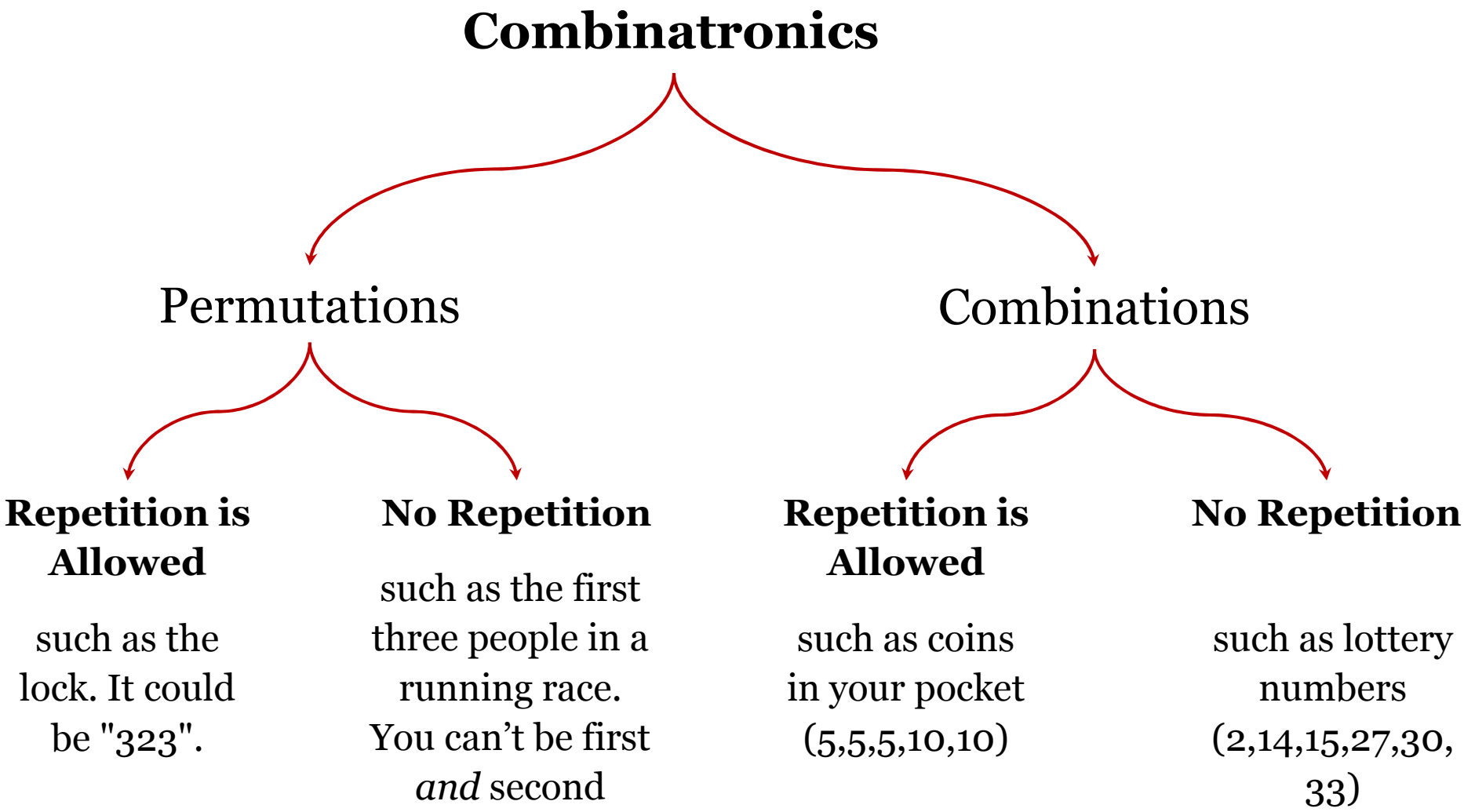
- If the order doesn't matter, it is a Combination.
- If the order does matter it is a Permutation.
- A Permutation is an ordered Combination.

My fruit salad is a combination of apples, grapes and bananas

The combination to the safe was 472

[6]

Combinatorics



Combinatorics

- **Permutations with Repetition**

  When you have ***n*** things to choose from … you have ***n*** choices each time!

  Number of permutations=n^r

  where

  n is the number of things to choose from, and you choose ***r*** of them (Repetition allowed, order matters)

Example: in the 3-digit lock, there are 10 numbers to choose from (0,1,..9) and you choose 3 of them:

10 × 10 × … (3 times) = 10^3 = 1,000 permutations

For Reading

[6]

Combinatorics

- **Permutations without Repetition**

  In this case, you have to **reduce** the number of available choices each time.

  Number of permutations= $\dfrac{n!}{(n-r)!}$

  $$P(n,r) = {}^{n}P_{r} = {}_{n}P_{r} = \frac{n!}{(n-r)!}$$

  where ***n*** is the number of things to choose from, and you choose ***r*** of them (No repetition, order matters)

Example: order of 3 out of 16 pool balls

$$\frac{16!}{(16-3)!} = \frac{16!}{13!} = 3{,}360$$

For Reading

[6]

Combinatorics

- **Combinations without Repetition**

  This is how **lotteries** work. The numbers are drawn one at a time, and if you have the lucky numbers (no matter what order) you win!

  Number of combinations= $\binom{n}{r} = \frac{n!}{r!(n-r)!}$

  $$C(n,r) = {}^{n}C_{r} = {}_{n}C_{r} = \binom{n}{r} = \binom{n}{n-r} = \frac{n!}{r!(n-r)!}$$

  where ***n*** is the number of things to choose from, and you choose ***r*** of them (No repetition, order doesn't matter)

  It is often called **"n choose r"** (such as "16 choose 3') and is also known as the **"Binomial Coefficient"**

For Reading

[6]

Combinatorics

- **Combinations without Repetition**

  In poll balls example, choose 3 out of 16 balls without order

  16-choose-3 is

$$C(16,3) = \binom{16}{3} = \frac{16!}{3!(16-3)!} = 560$$

For Reading

[6]

Combinatorics

- **Combinations with Repetition**

  Let us say there are five flavours of ice cream: **banana, chocolate, lemon, strawberry and vanilla**.

  You can have three scoops. How many variations will there be?

  There are **n=5** things to choose from, and you choose **r=3** of them.

  Order does not matter, and you **can** repeat!)

For Reading

[6]

Combinatorics

- **Combinations with Repetition**

  Let's use letters for the flavours: {b, c, l, s, v}. Example selections would be:

  ◇ {c, c, c} (3 scoops of chocolate)

  ◇ {b, l, v} (one each of banana, lemon and vanilla)

  ◇ {b, v, v} (one of banana, two of vanilla)

For Reading

[6]

Combinatorics

- **Combinations with Repetition**

Number of combinations=

$$\binom{n+r-1}{r} = \binom{n+r-1}{n-1} = \frac{(n+r-1)!}{r!(n-1)!}$$

where ***n*** is the number of things to choose from, and you choose ***r*** of them (Repetition allowed, order doesn't matter)

You can have three scoops. How many variations will there be?

$$= \frac{(5+3-1)!}{3!(5-1)!} = \frac{7!}{3!\times 4!} = 35$$

For Reading

[6]

Combinatorics

- **Combinatorial Problems**

  Combinatorial problems are class of problems in which the **decision variables** are discrete -i.e. where the **solution** is a **set (combination)**, or a **sequence (permutation)**, of **integers** or other **discrete objects**.

[7]

Combinatorics

- **Combinatorial Problems: TSP**

  Given n cities, a travelling salesman must visit the n cities and return home, making a loop (roundtrip). He would like to travel in the most efficient way (cheapest way or shortest distance or some other criterion).

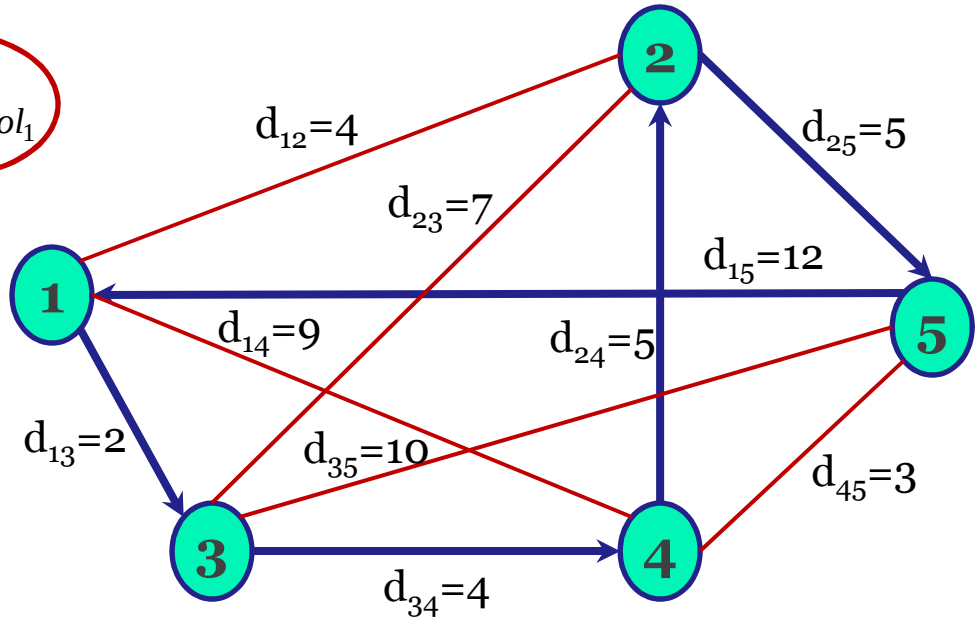
  Find the route which minimizes

$$total\_dist = \sum_{i=1}^{n-1} d_{sol_i} d_{sol_{i+1}} + d_{sol_n} d_{sol_1}$$

For a closed tour

The solution represented by permutation

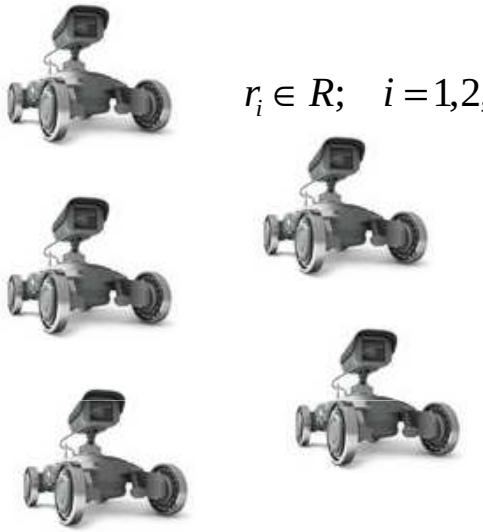
Sol = [1, 3, 4, 2, 5]



Combinatorics

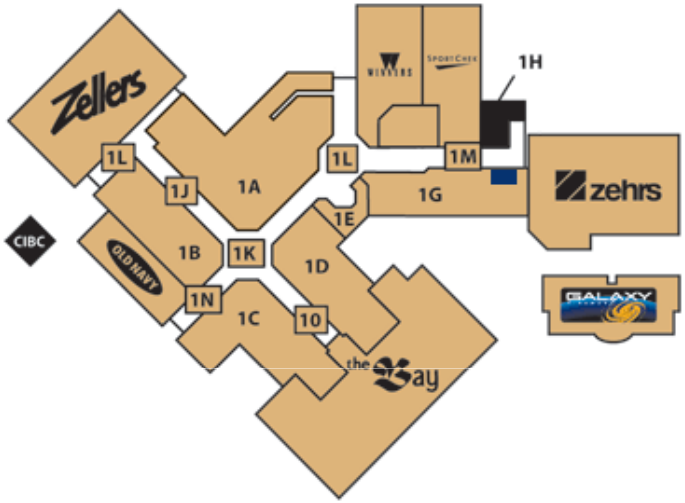
- **Combinatorial Problems: Assignment Problem**

$t_j \in T; \quad j = 1,2,...,m$



$r_i \in R; \quad i = 1,2,...,n$

$\pi : T \rightarrow R$



A set of n mobile Sensors: R

A set of m surveillance tasks: T

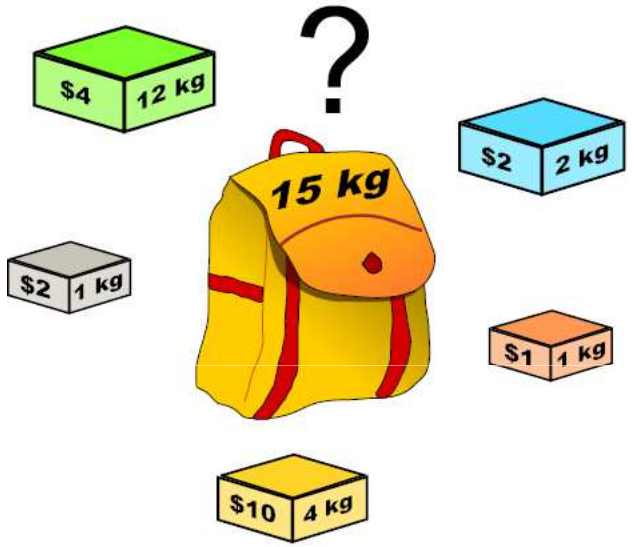
A set of n mobile sensors to carry out m surveillance tasks. If the sensor i does task j, it costs c_{ij} units. Find as assignment π which minimizes $\sum_{i=1}^{n}\sum_{j=1}^{m} c_{ij}\pi_{ij}$

The solution represented by permutations

Combinatorics

- **Combinatorial Problems: Bin Packing Problem**

Given n items, each with a weight w_i and a value u_i, to be packed into a bag of capacity C.



Determine the subset I of items which should be packed in order to maximize

$$\sum_{i \in I} u_i$$

such that

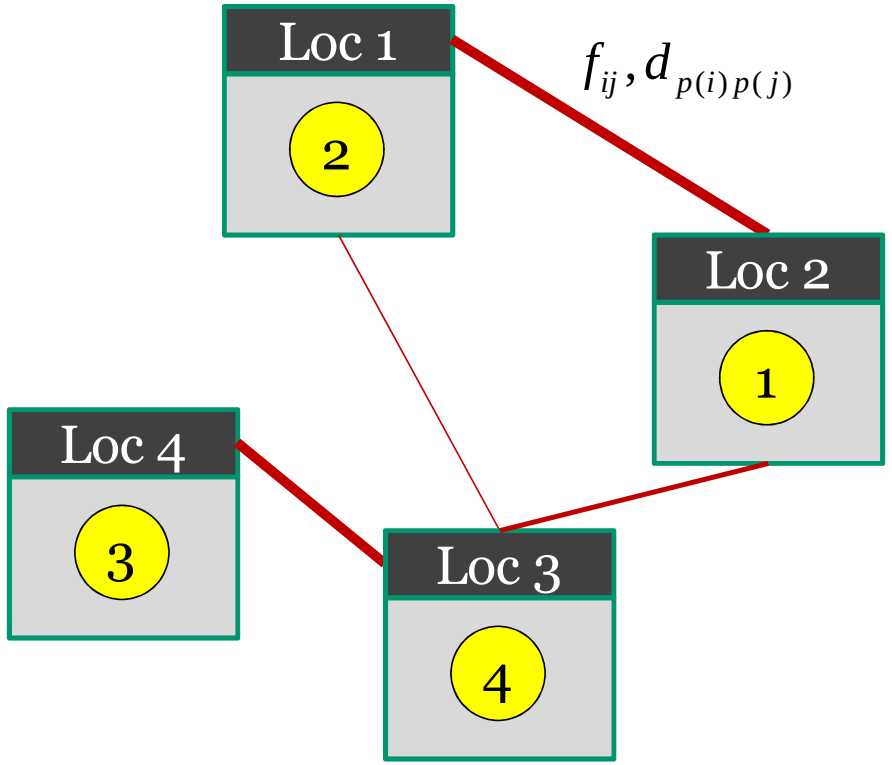
$$\sum_{i \in I} w_i \leq C$$

The solution is represented by a combination

Combinatorics

- **Combinatorial Problems: Plant Layout Problem (PLP)**

  Given n facilities (departments) and n locations, PLP aims at assigning different facilities (departments) to different locations in order to minimize the total material handling cost.



$$TMHC = \sum d_{ij} \times f_{ij}$$

The solution is represented by a combination of pairs.

Outline

- Introduction to Optimization
- Optimization and Operations Research
- Optimization Problem
- Optimization Problem Classifications
- Optimization Methods
- Combinatorics
- **Computational Complexity [For Reading]**

Computational Complexity

- **Algorithm Efficiency**
  - ◇ Run-time
  - ◇ Memory requirements
  - ◇ Number of elementary computer operations to solve the problem in the worst case. Examples of these **primitive operations** include evaluating an expression, assigning a value to a variable, indexing into an array, calling a method and returning from a method.

Computational Complexity

- **Algorithm Efficiency (cont'd)**
  - ***Example of primitive operations***

```
Algorithm arrayMax(A,n)
        currentMax←A[0]
        for i ←1 to n-1 do
                if A[i]>currentMax then
                        currentMax ←A[i]
        {increment i}
        Return currentMax
```

#of operations
1
n-1
n-1
n-1
1

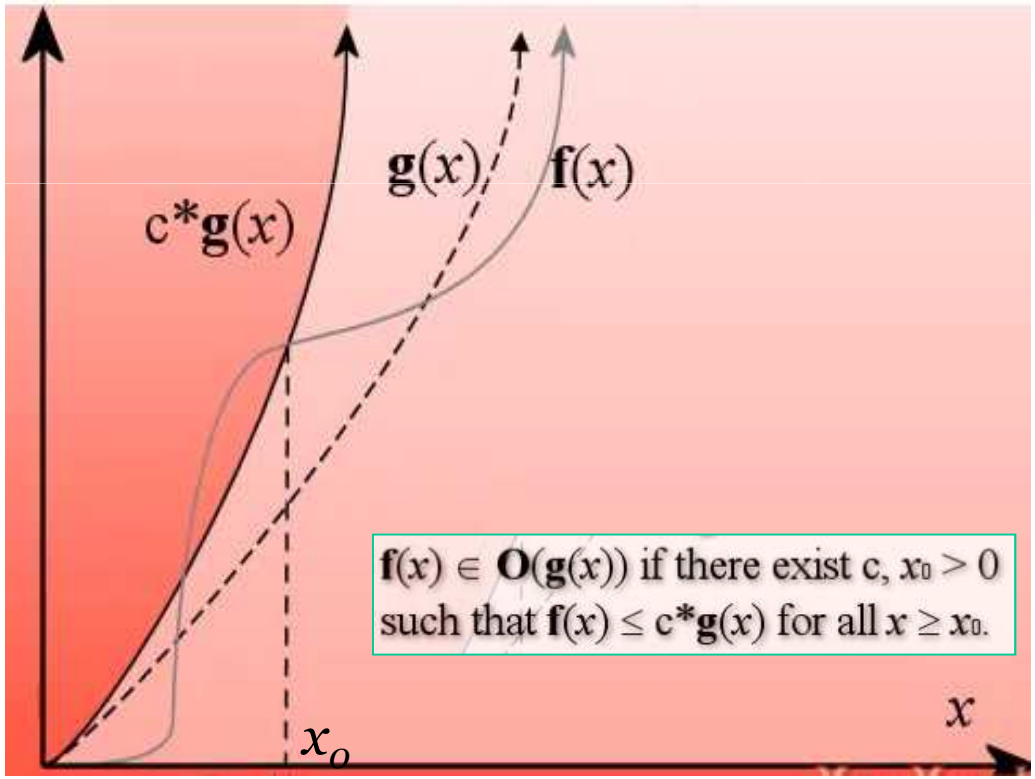
Worst case: 3n-1

Computational Complexity

- **Big "O" Notation**

  Big O notation is used in Computer Science to describe the **performance or complexity** of an algorithm.

  Big O specifically describes the **worst-case** scenario, and can be used to describe the execution time required or the space used (e.g. in memory or on disk) by an algorithm.



YouTube http://www.youtube.com/watch?v=6Ol2JbwoJpo

Computational Complexity

- **Big "O" Notation**

  The O notation for a function $f(x)$ is derived by the following simplification rules:

  ◇ If $f(x)$ is a **sum** of several terms, the one with the **largest growth rate** is kept, and all others omitted.

  ◇ If $f(x)$ is a **product** of several factors, any constants (terms in the product that do not depend on x) are omitted.

Computational Complexity

- **Big "O" Notation**

  Example-1: $f(x) = 6x^4 - 2x^3 + 5$

  ◇ This function is the sum of three terms: $\mathbf{6x^4}$, $\mathbf{-2x^3}$, and $\mathbf{5}$.

  ◇ Of these three terms, the one with the highest growth rate is the one with the largest exponent as a function of x, namely $\mathbf{6x^4}$.

  ◇ Now one may apply the second rule: $\mathbf{6x^4}$ is a product of 6 and $\boldsymbol{x^4}$ in which the first factor does not depend on x.

  ◇ Omitting this factor results in the simplified form x^4. Thus, we say that $\boldsymbol{f(x)}$ is a big-oh of $(\boldsymbol{x^4})$ or mathematically we can write $\boldsymbol{f(x) \in O(x^4)}$.

Computational Complexity

- **Big "O" Notation**

Example-2: $100n + 1000n^2 + 0.01n^3$

$$\Downarrow$$

$$O(n^3)$$

◇ Usually interested in performance on large instances.

◇ Only consider the dominant term (worst case scenario).

Computational Complexity

- **Big "O" Notation**

  Example-3: Simplify the following order expression to the closest orders.

  a) $0.9n + 15$;

  b) $1.5n^3 + 2n^2 + 10n$;

  c) $n\log(n)+\log(2n)$;

  Solution:

  a) $O(n)$,

  b) $O(n^3)$,

  c) $O(n\log n)$

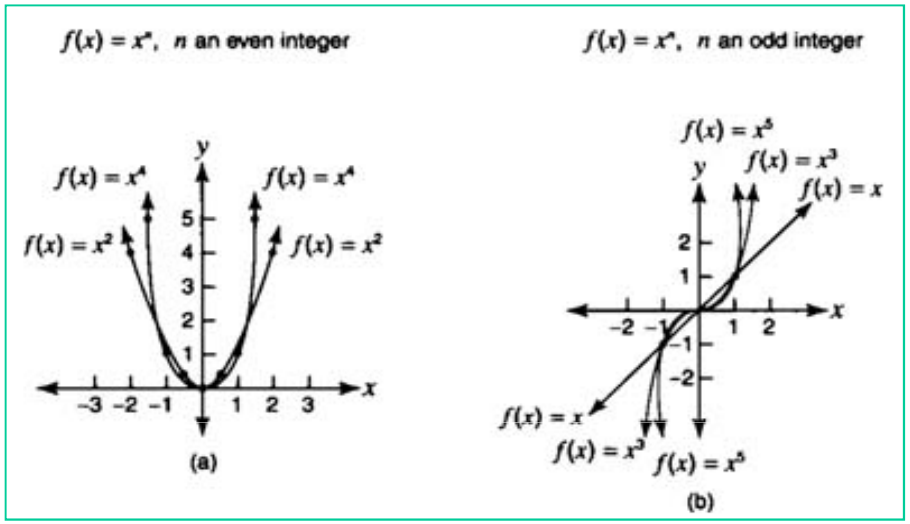
Computational Complexity

- **Big "O" Notation**

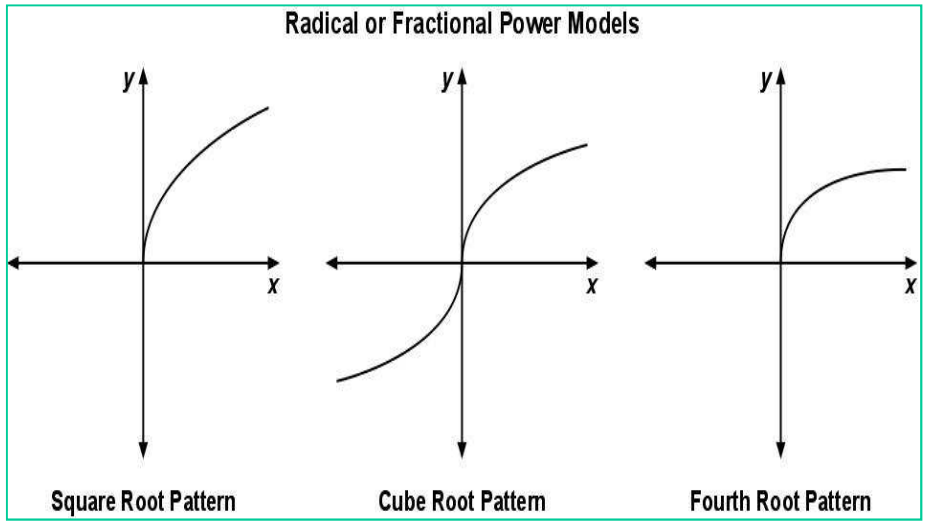
Notation	Name
O(1)	Constant
O(logn)	Logarithmic
$O(n^c)$, 0<c<1	Fractional power
O(n)	Linear
O(nlogn)=O(logn!)	Linearithmic, loglinear or quasilinear
$O(n^2)$	Quadratic
$O(n^c)$, c>1	Polynomial or algebraic
$O(c^n)$, c>1	Exponential
O(n!)	Factorial

Computational Complexity

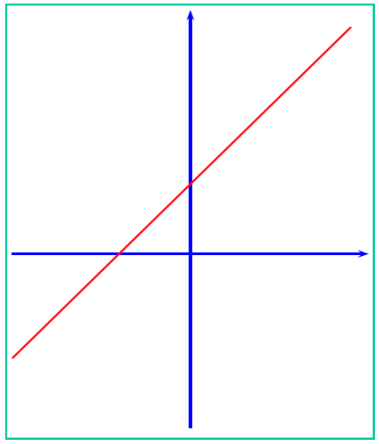
- **Big "O" Notation**



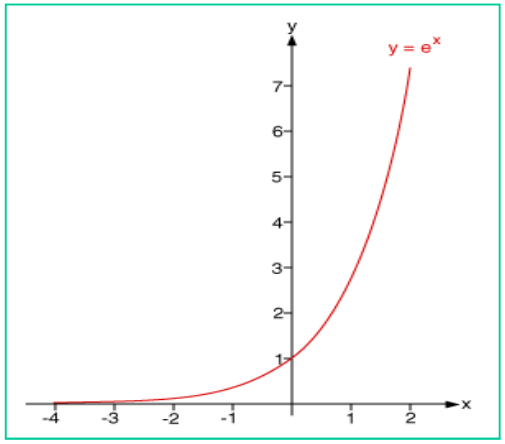
Polynomial function



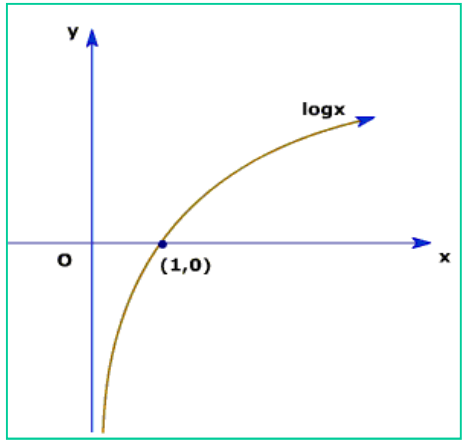
Fractional power function



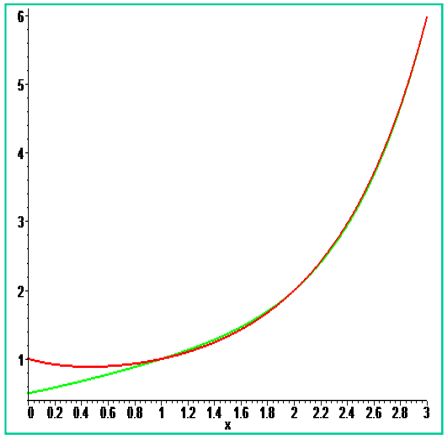
Linear function



Exponential function



Logarithmic function



Factorial function

Computational Complexity

- **Big "O" Notation**

Example-1:

O(1): describes an algorithm that will always execute in the same time (or space) **regardless of the size** of the input data set.

```
bool IsFirstElementNull(String[] strings)
{
        if(strings[0] == null)
        {
                return true;
        }
        return false;
}
```

[8]

Computational Complexity

- **Big "O" Notation**

Example-2:

O(N) describes an algorithm whose performance will grow linearly and in direct proportion to the size of the input data set.

Big O favors the worst-case performance scenario.

```
bool ContainsValue(String[] strings, String value)
{
        for(int i = 0; i < strings.Length; i++)
        {
                if(strings[i] == value)
                {
                        return true;
                }
        }
        return false;
}
```

[8]

Computational Complexity

- **Big "O" Notation**

Example-3:

O(N²) represents an algorithm whose performance is directly proportional to the square of the size of the input data set. This is common with algorithms that involve nested iterations over the data set. Deeper nested iterations will result in O(N³), O(N⁴) etc.

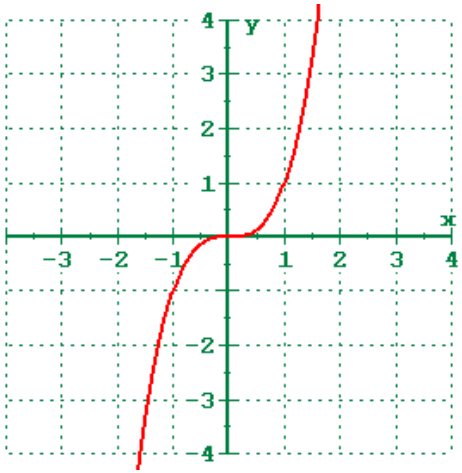
```
bool ContainsDuplicates(String[] strings)
{
        for(int i = 0; i < strings.Length; i++)
        {
                for(int j = 0; j < strings.Length; j++)
                {
                        if(i == j) // Don't compare with self
                        {
                                continue;
                        }

                        if(strings[i] == strings[j])
                        {
                                return true;
                        }
                }
        }
        return false;
}
```

[8]

Computational Complexity

- **Polynomial vs Exponential-Time Algorithms**
  - ◇ What is a "**good**" algorithm?
  - ◇ It is commonly accepted that worst case performance bounded by a **polynomial function** of the problem parameters is "good". We call this a **Polynomial-Time Algorithm**

Example: $O(n^3)$



  - ◇ Strongly preferred because it can handle arbitrarily large data.

Computational Complexity

- **Polynomial vs Exponential-Time Algorithms**
  - ◇ In **Exponential-Time Algorithms**, worst case run time grows as a function that cannot be polynomially bounded by the input parameters.

    Example: $O(2^n)$ $O(n!)$
  - ◇ **Q:** Why is a polynomial-time algorithm better than an exponential-time one?

    A: Exponential time algorithms have an **explosive growth rate**.

Computational Complexity

- **Polynomial vs Exponential-Time Algorithms**

Big-oh/size	n=5	n=10	n=100	n=1000
n	5	10	100	1000
n^2	25	100	10000	1000000
n^3	125	1000	1000000	10^9
2^n	32	1024	$1.27x10^{30}$	$1.07x10^{301}$
n!	120	$3.6x10^6$	$9.33x10^{157}$	$4.02x10^{2567}$

Computational Complexity

- **Problem Classes**

Complexity class	Model of computation	Resource constraint	Complexity class
DTIME(f(n))	Deterministic Turing machine	Time f(n)	DTIME(f(n))
P	Deterministic Turing machine	Time poly(n)	P
NP	Non-deterministic Turing machine	Time poly(n)	NP

Computational Complexity

- **Problem Classes**
  - ◇ In theoretical computer science, a **Turing machine** is a theoretical machine that is used in thought experiments to examine the abilities and limitations of computers.
  - ◇ In essence, a Turing machine is imagined to be a simple computer that reads and writes symbols one at a time on an endless tape by strictly following a set of rules. It determines what action it should perform next according to its internal "state" and what number it currently sees.

    An example of one of a Turing Machine's rules might thus be: "**If you are in state 2 and you see an 'A', change it to a 'B' and move left**."

Source: http://en.wikipedia.org/wiki/Turing_machine

Computational Complexity

- **Problem Classes**
  - ◇ In a **deterministic Turing machine**, the set of rules prescribes at most one action to be performed for any given situation.
  - ◇ A **non-deterministic Turing machine (NTM)**, by contrast, may have a set of rules that prescribes more than one action for a given situation. For example, a non-deterministic Turing machine may have both "**If you are in state 2 and you see an 'A', change it to a 'B' and move left**" and "**If you are in state 2 and you see an 'A', change it to a 'C' and move right**" in its rule set.

Source: http://en.wikipedia.org/wiki/Turing_machine

Computational Complexity

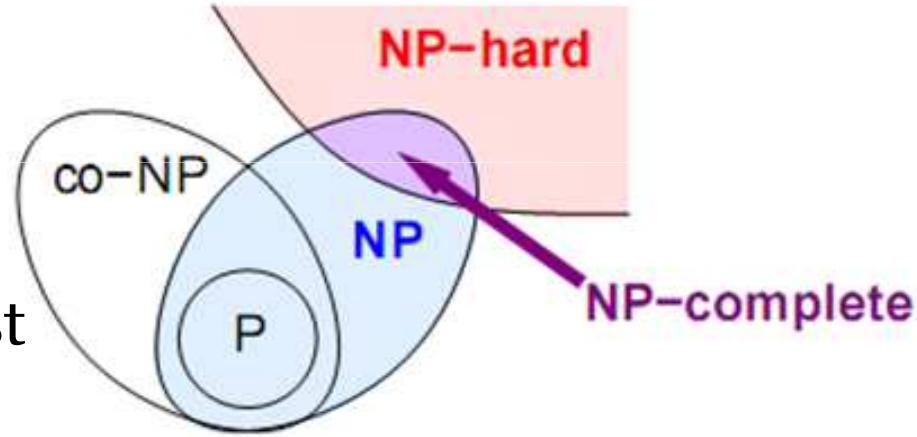
- **Problem Classes**
  - ◇ **P**
    - Deterministic in nature
    - Decision problems (decision problems) that can be solved in polynomial time ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ or $O(n^2)$)
    - Can be solved "efficiently" i.e. P includes all decision problems for which there is an algorithm that halts with the correct yes/no answer in a number of steps bounded by a polynomial in the problem size n.
    - The Class P in general is thought of as being composed of relatively "easy" problems for which efficient algorithms exist.

Computational Complexity

- **Problem Classes**
  - ◇ **NP**
    - NP = Nondeterministic Polynomial
    - Decision problems whose "**YES**" answer can be verified in polynomial time, if we already have the proof (or witness)
    - If someone tells us the solution to a problem, we can verify it in polynomial time, i.e. easy to **verify** but not necessarily easy to **solve.**
  - ◇ **co-NP**
    - Decision problems whose "**NO**" answer can be verified in polynomial time, if we already have the proof (or witness)

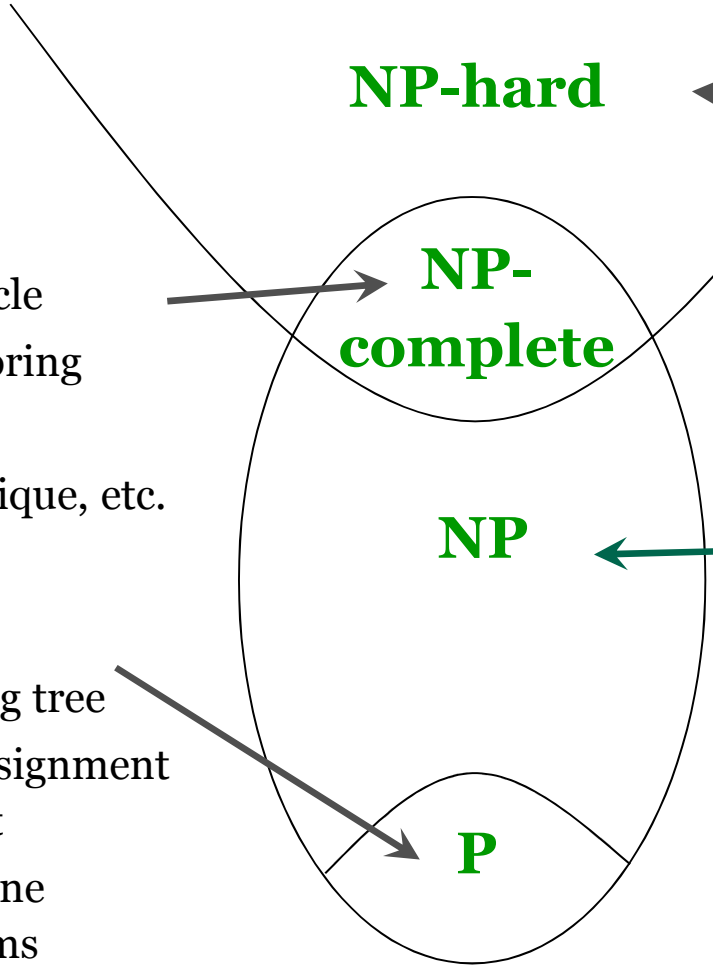
Computational Complexity

- **Problem Classes**
  - ◇ **NP-hard**
    - – A problem is NP-hard **iff** a polynomial-time algorithm for it implies a polynomial-time algorithm for every problem in NP
    - – NP-hard problems are at least *as hard as* NP problems
  - ◇ **NP-complete**
    - – A problem is NP-complete if it is NP-hard, and is an element of NP (NP-easy)



Computational Complexity

- **Problem Classes**



- TSP
- Graph coloring
- Set covering and partitioning
- Knapsack
- Precedence-constrained scheduling, etc.

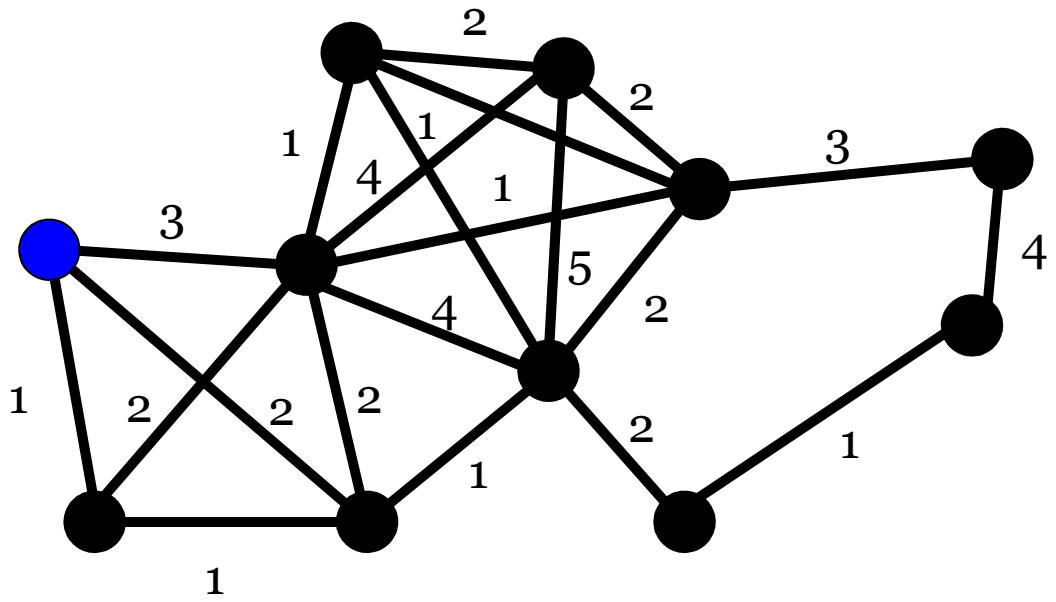
- N-puzzle
- Hamilton cycle
- Graph 3-coloring
- Satisfiability
- Maximum clique, etc.

- Factoring
- Graph isomorphism
- Minimum circuit size
- Virtually all the widely studied combinatorial optimization problems.

- Shortest path
- Minimum spanning tree
- Transportation, assignment and transshipment
- Some single machine scheduling problems
- Matrix determinant

Computational Complexity

- **Problem Examples:** Travelling Salesman Problem (TSP)



11!=39,916,800 different ways

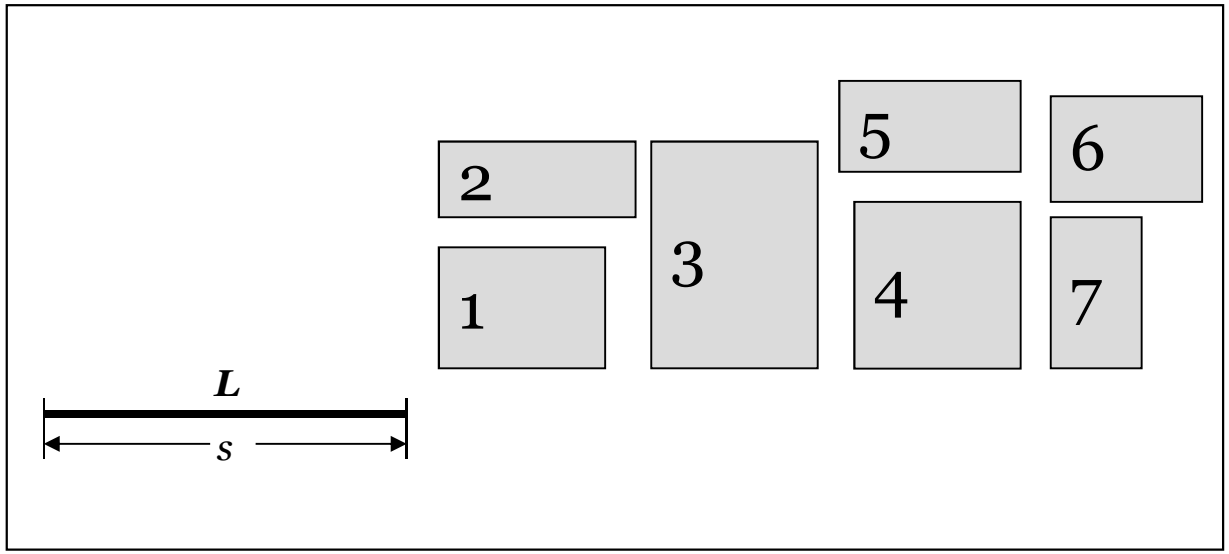
cost = 23

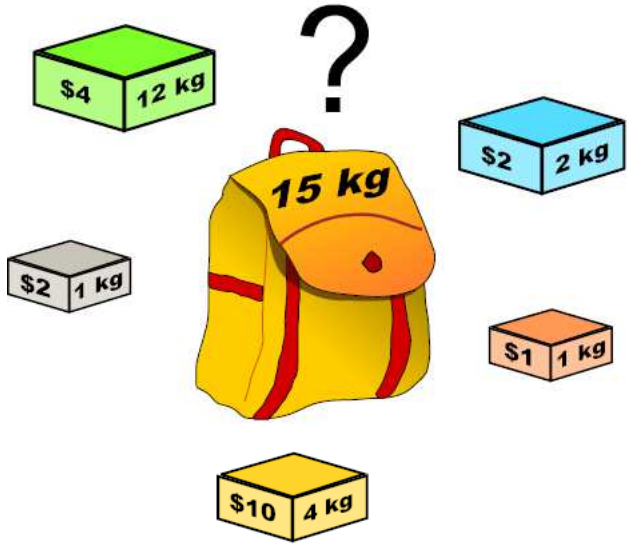
For each two cities, an integer cost is given to travel from one of the two cities to the other. The salesperson wants to make a minimum cost circuit visiting each city exactly once.

TSP is an NP-hard problem in combinatorial optimization.

Computational Complexity

- **Problem Examples:**





Knapsack Problem

Given s and w can we translate a subset of rectangles to have their bottom edges on L so that the total area of the rectangles touching L is at least w?

This is a combinatorial NP-hard problem.

Bin Packing Problem

Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

This is a combinatorial NP-hard problem.

References

1. Andres Ramos. Operations Research and Optimization. Universidad Pontificia Comillas, 2010.
2. Singiresu S. Rao. Engineering Optimization: Theory and Practice. John Wiley & Sons, 2009.
3. Ruey-Maw Chen and Hsiao-Fang Shih, "Solving University Course Timetabling Problems Using Constriction Particle Swarm Optimization with Local Search", Algorithms 2013, 6, 227-244; doi:10.3390/a6020227.
4. A. Engelbrecht. Fundamentals of Computational Swarm Intelligence. Wiley, 2005.
5. CSP Tutorial, Cork Constraint Computation Centre (4C): http://4c.ucc.ie/web/outreach/index.html. Last accessed: May 7, 2014.
6. http://www.mathsisfun.com/combinatorics/combinations-permutations.html Last accessed: May 7, 2014.
7. C. Revees. *Modern Heuristic Techniques for Combinatorial Problems*. Halsted Press, New York, 1993.
8. Rob Bell. A Beginner's Guide to Big O Notation, http://rob-bell.net/2009/06/a-beginners-guide-to-big-o-notation. Last accessed: May 7, 2014.